

LodusPop: An Environment-Driven Multi-Scale Simulation Framework for Urban Population Dynamics

Gabriel Fonseca Silva, André da Silva Antonitsch and
Soraia Raupp Musse

*Virtual Humans Laboratory (VHLab) - School of Tecnology, Pontifical Catholic University of Rio Grande do Sul (PUCRS), Av. Ipiranga, 6681
- Partenon, Porto Alegre, 90619-900, RS, Brazil.

*Corresponding author(s). E-mail(s): fonseca.gabriel@pucrs.br;
Contributing authors: andre.antonitsch@edu.pucrs.br;
soraia.musse@pucrs.br;

Abstract

Urban environments face persistent challenges in managing mobility and public health, creating a growing demand for advanced simulation tools to support decision-making. Existing population-driven frameworks often require detailed individual data, raising privacy concerns and limiting scalability. Moreover, most approaches are constrained to a single modeling scale, making it difficult to capture both individual routines and collective urban dynamics within one framework. This paper introduces LodusPop, a flexible multi-scale simulation framework for urban population dynamics. LodusPop employs an environment-driven paradigm, where environments actively request population movements, reducing dependence on fine-grained, individual behavioral data. Populations are represented as blobs, aggregated groups of arbitrary size with defined characteristics, that can split, merge, and adapt to changing simulation conditions. This abstraction balances computational efficiency with behavioral expressiveness, enabling the study of diverse urban phenomena across scales ranging from neighborhoods to entire cities. We demonstrate LodusPop's versatility through three case studies in Porto Alegre, Brazil, addressing everyday mobility, large-scale events, and epidemic control with vaccination policies. The results highlight LodusPop as a practical and extensible tool for urban planning, crisis management, and public health interventions, bridging the gap between data availability,

computational scalability, and the ability to analyze population dynamics across multiple spatial and temporal scales.

Keywords: Environment-driven simulation, Multi-scale modeling, Human mobility, Crowd simulation, Urban population dynamics

1 Introduction

Recent crises have underscored the critical role of population modeling in urban planning and crisis management. Cities worldwide have faced urgent challenges in managing mobility, enforcing social distancing measures, and regulating the use of public spaces such as malls, supermarkets, and health services[1–3]. Similar challenges also emerge in other contexts, including disaster response, large-scale events, and climate-related risks[4–7]. These scenarios highlight the importance of simulation systems that can support the evaluation of urban policies and decisions, providing insights into how individuals and groups interact with the built environment[2, 5].

Simulation models are central to supporting evidence-based urban planning, from optimizing public transportation and allocating residential areas to improving the overall resilience of city systems [4, 5]. Human mobility is particularly relevant, forming the basis of research in epidemic control, traffic simulation, and socio-economic studies [6, 7]. Existing methods span microscopic approaches that track individual agents [8–11], macroscopic approaches that capture aggregated flows [12–14], and hybrid models that combine both perspectives [15–18]. In this paper, our focus is on urban population modeling and mobility-oriented simulations across these scales, rather than on high-fidelity, collision-level crowd simulation. Despite the significant advances of existing methods, two major challenges remain for such population-dynamics platforms: i) integration across scales—connecting individual-level routines with population-wide patterns in a consistent framework [19, 20]; and ii) dependence on detailed data—most population-driven models rely on granular behavioral or mobility data, which is often unavailable or impractical to collect due to privacy and scalability concerns [19, 20].

To address these challenges, we introduce LodusPop, a flexible framework for simulating urban populations across multiple spatial and temporal scales. Instead of focusing on individual decision-making and detailed trajectory planning, LodusPop targets macro- and meso-scale population dynamics: environments actively request movements from aggregated population units. This environment-driven paradigm reduces reliance on fine-grained data while still capturing key patterns in aggregate flows and occupancy. Populations are represented through blobs – arbitrarily sized groups of individuals with defined attributes and routines – that can split, merge, and evolve during a simulation. This abstraction balances computational efficiency with the ability to represent heterogeneous population behaviors across scales ranging from neighborhoods to entire cities, while still providing hooks for micro-style routines when localized detail is needed.

The main contributions of this paper are as follows:

- We propose an environment-driven modeling paradigm that shifts decision-making from individuals to the environment, reducing data requirements while remaining suitable for multi-scale urban mobility and policy analysis.
- We introduce the blob abstraction, enabling flexible and scalable representation of populations that adapt to data availability and simulation needs.
- We design LodusPop, a multi-scale simulation framework that integrates GIS-based environments with a blob-based population model within a unified architecture.
- We demonstrate LodusPop’s versatility through three case studies in Porto Alegre, Brazil, covering urban mobility, large-scale events, and epidemic control with vaccination policies.

Together, these contributions establish LodusPop as a practical and extensible framework for studying urban population dynamics, offering valuable support for decision-making in both routine planning and emergency management.

2 Related Work

Research on modeling population dynamics spans two central but historically distinct areas: crowd simulation and population mobility modeling. Crowd simulation emphasizes fine-grained representations of movement and interactions among individuals or groups, whereas population mobility focuses on larger-scale flows and statistical regularities in urban dynamics. Each domain has produced significant advances, but both exhibit limitations that motivated the development of LodusPop.

2.1 Crowd Simulation

Crowd simulation has long served as a foundation for understanding human movement in both indoor and outdoor environments, with models developed at microscopic, macroscopic, and hybrid scales.

Microscopic models treat every agent as an autonomous entity with decision-making capabilities, typically influenced by local environmental conditions. Classical examples include the Social Force Model [8], where pedestrian motion emerges from forces balancing personal goals with environmental and social interactions, and ORCA [21], which guarantees collision-free trajectories through reciprocal velocity obstacles. BioCrowds [22] introduced a space-discretization strategy where agents compete for territory, naturally generating emergent Voronoi-like partitions. Later studies aimed to add heterogeneity by embedding psychological or emotional states into agent behavior. For example, Pelechano et al. [23] and Durupinar et al. [24] integrated factors such as stress, emotion, and cultural variation, while Neto et al. [25] modeled emotional contagion based on personality traits. These approaches achieve remarkable behavioral variety, but they depend heavily on detailed behavioral datasets and quickly become computationally expensive when scaled up to large urban populations.

In contrast, macroscopic models treat crowds as collective entities, often described through density fields or flow equations. Hughes [12] and Treuille et al. [13] pioneered continuum models where population density evolves through potential fields defined over the environment, while proposed Antonitsch et al. [17] BioClouds, aggregating

agents into homogeneous “clouds” competing for space based on density and velocity preferences. Such formulations shift part of the steering to environment-side representations and significantly improve computational scalability, but they sacrifice the diversity of individual behaviors.

Hybrid models attempt to combine these two perspectives. For example, Legion[26] dynamically adapts the level of detail based on environmental complexity, allowing legions of agents to split or merge as situations demand. Xiong et al. [15] proposed spatially partitioned hybrid simulations that combine ORCA [21] in high-interest regions with continuum models elsewhere, while Lemonari et al. [27] introduced MPACT, a mesoscopic framework that uses learned profiles to capture intermediate-scale behaviors. These hybrids achieve flexibility, however, the primary focus of control remains on the population side. Agents query these fields and detailed data is often required for accurate calibration.

Recent advances explore ways to alleviate this dependency through data-driven modeling, where behavior is learned from observed or simulated trajectories rather than being entirely hand-crafted. For instance, machine learning has been used to embed agent dynamics into low-dimensional manifolds, enabling multiscale reconstruction of crowd flows [28]. Deep reinforcement learning approaches, such as those proposed by Li et al. [29], combine learned navigation strategies with anisotropic environmental fields to reduce computational costs in large spaces. In many of these studies, policies are learned from large numbers of simulated episodes rather than from real mobility datasets. Behavior is governed by fitted models whose data requirements center on simulated experience rather than fine-grained observational traces, and the underlying agent-based perspective remains unchanged.

In summary, crowd simulation excels in behavioral realism and the reproduction of fine-scale interactions. However, the reliance on detailed micro-data and high computational costs make it difficult to deploy at the scale of entire cities or to apply when granular behavioral data is unavailable.

2.2 Population Mobility

Population mobility studies emerged from statistical and large-scale analyses of human movement patterns. Unlike crowd simulation, which centers on the agent, mobility modeling typically abstracts away individuality to emphasize flows across geographic space and time.

A consistent finding in this field is that human displacements follow heavy-tailed distributions. Brockmann et al. [30] showed that the circulation of banknotes in the United States resembled Lévy flights, while González et al. [31], analyzing cell phone data, demonstrated that most individuals repeatedly visit a small set of locations. Song et al. [32] formalized this as a combination of exploration (discovering new places) and preferential return (frequent revisits), and Alessandretti et al. [20] refined these ideas with hierarchical “containers”, arguing that people conceptualize space in nested levels such as neighborhoods, cities, and regions. These statistical regularities provide strong predictive power, but they are generally limited to describing long-term patterns rather than simulating short-term interventions or local policies.

The COVID-19 pandemic highlighted the importance of connecting mobility with public health. Studies such as Jamshidi et al. [33] examined the effect of density and mobility restrictions on infection rates, while Hernández-Orallo et al. [34] used urban simulations to identify risk hotspots like train stations and plazas. Kraemer et al. [35] quantified how mobility and control measures influenced epidemic spread in China. These works demonstrated the societal relevance of mobility modeling but remained largely domain-specific, oriented toward epidemiology rather than more general urban planning applications.

Frameworks for simulating mobility have also emerged. Silva et al. [36] introduced a framework that integrates environmental data with macroscopic BioClouds to simulate contagion scenarios. Although innovative, its reliance on homogeneous clouds and a narrow focus on epidemics limited its applicability. More recent frameworks increasingly incorporate machine learning and multi-scale techniques [6,7,16], but they remain strongly tied to data availability and typically adhere to the population-driven logic of traditional mobility models.

Recent contributions push the field toward more interpretable, adaptable, and AI-enhanced approaches. Liu et al. [37] proposed GraviGBM, which blends gravity-based migration models with gradient-boosted machine learning, balancing predictive accuracy and interpretability. Other work has used symbolic regression to uncover interpretable mobility laws directly from data, identifying power-law patterns in flows [38]. On the simulation side, new agent-based frameworks integrate large language models with mobility environments, as in CAMS [39], enabling autonomous agents to generate plausible movement patterns without pre-defined geospatial inputs.

Additionally, several general-purpose urban simulation frameworks are widely used. MATSim [40] is a large-scale agent-based framework for transportation planning that simulates individual travel demand and traffic assignment through iterative optimization. SUMO [41] is a microscopic traffic simulator designed for modeling vehicles and road networks, offering fine-grained mobility but limited population abstraction. GAMA [42] provides a flexible multi-agent platform where users can model a wide range of urban and environmental processes with rich GIS integration.

In summary, mobility research provides robust statistical frameworks and promising AI-driven methods, but most approaches are either too aggregated for dynamic, multi-scale interventions or remain dependent on detailed mobility data.

2.3 Positioning LodusPop Between Crowd Simulation and Mobility Models

The works reviewed above point to a gap between crowd simulation and population mobility models. On one side, crowd simulators are capable of reproducing rich local interactions and complex behaviors, but they become expensive at city scale and often presuppose detailed, behavior-level data for calibration. Mobility models, in turn, describe flows at coarser levels and scale more easily to large regions, but offer limited support for representing how specific locations, policies, or events structure movement over time in a way that remains explicit and manipulable.

LodusPop is designed to bridge this gap by making the environment an explicit driver of population dynamics while retaining heterogeneity in the population. Routines are attached to regions and points of interest instead of individuals, so locations express which types of population they demand or release at different times. For example, schools requesting children from nearby residential areas, hospitals attracting patients with particular attributes, or shelters drawing specific profiles subject to capacity constraints. Populations are represented as blobs of variable size that carry typed, traceable attributes and can split when only a subpopulation participates in an action or undergoes a change of state, and merge again when compatible groups share the same location. In this way, a single simulation can move smoothly between aggregate flows and more detailed descriptions where fine tracking is important, without losing consistency of attributes over time.

The dynamics of LodusPop are expressed in a compact yet extensible action formalism that covers both movements in the environment and changes in traceable characteristics. Routines attached to locations are written as sequences of such actions, which the simulator interprets to update blobs and their distribution across the environment. Additional processes such as disease transmission, recovery, vaccination, or participation in specific events can be introduced as new action types together with their consumption rules, implemented as plugins that integrate with the same core abstraction. As a result, LodusPop can combine different mobility and process models within a unified environment-driven representation, reusing the same engine to explore diverse scenarios while maintaining a clear separation between the core simulator and domain-specific mechanisms.

3 Proposed Model

Population mobility area studies the large-scale flow of a population in a given geographic space, describing this population as a whole instead of its individuals [30, 31]. This approach may have different degrees of granularity, as populations can be studied in a given district, city, state, country, continent, or even the entire world and in different time scales, as data can be hourly, daily, monthly, yearly, and so on. In this work, we propose LodusPop, a flexible framework for multi-scale population simulation. Instead of representing individuals or fixed groups, LodusPop models populations as variable-sized units, each with defined characteristics and behavioral patterns. This design enables seamless adjustment of simulation detail, adapting to both the availability of data and the requirements of the study.

The simulation process relies on data to function effectively. LodusPop can utilize real-world datasets while offering the flexibility to customize population parameters. Indeed, LodusPop was designed to simulate population mobility allowing the integration of data from diverse sources, such as real-world datasets and programmed scripts. Figure 1 illustrates the potential data sources and their integration within the components of our proposed model. On the left, we can see possible sources of data, which are specified in the center, that can be used in the platform. On the right, the core components of LodusPop are depicted: the environment, defining the simulated space; the population, representing entities moving within the environment; and

the simulation behavior, responsible for generating actions that influence the population. These components are detailed in the subsequent sections. As an example, a simulation designer can define a population and model their behavior using a combination of real-world and synthetic datasets. Environmental data can be sourced from Geographic Information Systems (GIS) databases like OpenStreetMap. Population profiles may be derived from census data (Real Population) or generated synthetically (Synthetic Population) to fit the specific needs of an experiment. Additionally, data describing population behavior, such as routines, can be obtained from sources like Origin-Destination Matrices, GPS tracking services, or user-generated inputs.

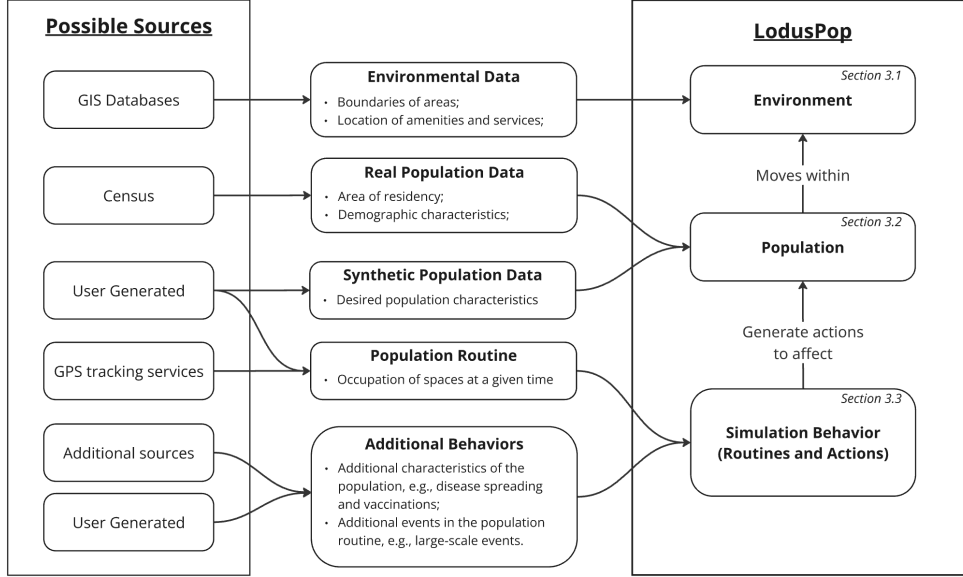


Fig. 1 Flowchart illustrating the integration of data sources into the LodusPop framework. Inputs such as GIS databases, census data, user-generated data, and GPS tracking services are processed into geographic data, population characteristics, and behavioral routines. In our proposed model, routines consist of actions that influence population behavior, driving their movement through the environment.

Population mobility frameworks typically operate across multiple spatial scales, such as neighborhoods, cities, and states. In this work, we seek to integrate strategies from crowd simulation into human mobility modeling, particularly the ability of macroscopic models to abstract individual behaviors [13, 14]. To achieve this, we propose representing populations as blobs—aggregated groups of individuals of varying sizes. In this context, Blob is an abstract structure, similar to a cloud or a legion [17], but designed to represent larger and more heterogeneous populations. Unlike Bio-Clouds and Legion, which assume homogeneous population clusters, our blobs can encompass diverse individual and group attributes and evolve within environments containing locations that demand specific population profiles.

Furthermore, our environment design builds upon the approach proposed by Silva et al. [36]. However, our model introduces greater flexibility, allowing regions of the simulated environment to represent different semantic entities beyond the fixed hierarchy of countries, states, regions, and cities, enabling more adaptable and context-specific representations.

In LodusPop, we can define a set of characteristics for each blob, representing a population of a specific profile distributed in the environment and changing temporally and spatially in the simulation. For instance, one group, representing a blob, might consist of children going to school in the morning and returning home in the afternoon, illustrating spatial movement within a city. Another example involves changes in population parameters, such as healthy workers commuting to their jobs in the morning, with some individuals becoming sick during the day, resulting in healthy individuals returning home and sick individuals heading to a hospital, in this case representing a split in the blob structure.

Our proposed model considers the following elements to provide population simulation, which will be described in the next sections.

- a) An environment modeled as a graph of regions and points of interest that the population can occupy;
- b) The population is modeled as blobs that can split and merge and can contain characteristics of the population; and
- c) a set of population movement actions and routines to describe the desired population behavior during the simulation.

3.1 Environment

To adequate our population mobility simulation to different spatial scopes, we model the environment as an abstract graph composed of *regions* and *points of interest*. The regions (R) represent areas of physical space in the virtual world that can contain points of interest (P), which represent locations that motivate human travel and that can have population groups (i.e., blobs B). When setting up a simulation, users define the spatial scope that R and P represent. For example, in a city simulation, R could correspond to districts, while P might represent markets, malls, hospitals, residential buildings, and commercial centers.

A region R is represented as a set of points of interest $\{P_1, \dots, P_m\}$. Each point of interest P_i stores the blobs currently located there, denoted $\mathbf{B}_{P_i}$ and a local routine Ro_{P_i} that specifies which actions occur at that location over time as the simulation evolves (3.3). Table 1 summarizes the notation used for regions and points of interest. Additionally, Appendix A outlines the process of defining the input data necessary for designing a simulation. Appendix A.1 specifically addresses the data required to model the simulation environment.

Indeed, this characterizes an inversion of responsibility compared to what usually happens in population simulations. In many agent-based approaches, populations decide where to go based on individual preferences or probabilistic rules, requiring granular data about individual movements and locations. Even in field-based models, where motion is guided by environmental potentials or vector fields, agents typically

remain the entities that sample and follow these fields. In our case, the environment is responsible for decision-making regarding the occupation of the population during the simulation, i.e., locations ask for people instead of people choosing where to go. We believe that shifting the focus to the environment as the decision-maker minimizes dependence on granular individual-level data. This shifts part of the decision logic from individuals to points of interest and regions, and is designed to minimize dependence on granular individual-level data by leveraging aggregated or generalized information about population distributions and environmental conditions. For example, a P_m representing a market can request consumers, i.e., P_m has a local routine Ro_{P_m} , which defines that a certain population of determined characteristics will go to the market during the simulation. B_{P_m} represents all the population blobs that are currently on the market.

Other examples in the same region R can be a school that requests students, and hospitals that request patients. In these cases, school is a P_j in R , in which Ro_{P_j} is the local routine to go to the school and B_{P_j} is a specific population of students. Similarly, the hospital is also a P_k in R , and Ro_{P_k} is a local routine that should be executed at the hospital. Additionally, B_{P_k} is a particular population that goes to the hospital. Figure 2 presents a diagram of a LodusPop environment, representing regions, points of interest, and blobs. As humans generally tend to present cyclical behavior in their movement [31], repeating similar routines daily or weekly, a LodusPop simulation is also cyclical, where local routines Ro repeat every time cycle of simulation. The user defines the temporal scope of a cycle when designing the simulation. A cycle may represent an hour, a day, a month, etc.

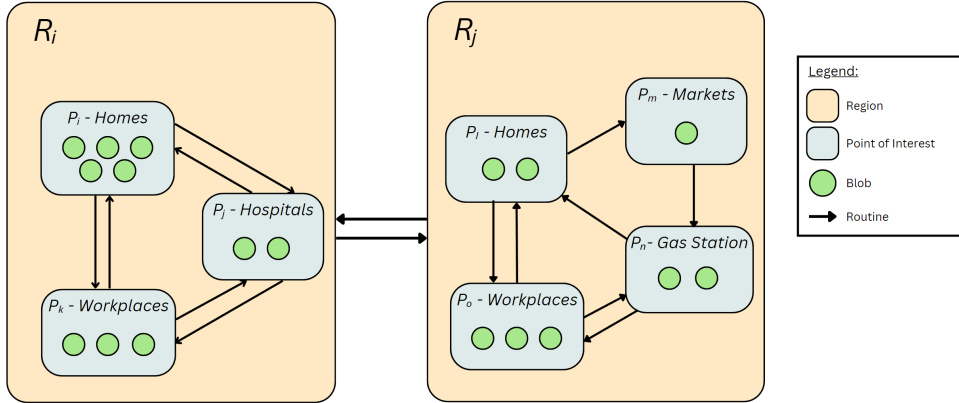


Fig. 2 A diagram of a LodusPop environment. The figure presents two regions, R_i and R_j , containing different points of interest P . Each point of interest may contain groups of the population (i.e., blobs), and routines are responsible for moving them in the environment during the simulation.

Table 1 Mathematical notation and descriptions of the elements defined in Section 3.1.

Element	Notation	Description
Region	R	Represent areas of physical space in the virtual world that can contain points of interest. A region R_i is defined as a set of P .
Point of Interest	P	Represent locations that the population can occupy as they move during a simulation. A P_i is defined by a collection of population blobs (B_{P_i}) and its local routine (Ro_{P_i})

3.2 Population

The population in LodusPop is represented by an abstraction of people into blobs B , each always located at some point of interest. These blobs describe the population in groups rather than tracking individuals. Formally, we denote a blob as $B = (TP, \mathbf{Tc}, \mathbf{Sc})$ where TP is the total number of individuals in the blob, $\mathbf{Tc}$ is the collection of traceable characteristics, and $\mathbf{Sc}$ is the collection of sampled characteristics. Table 2 summarizes this notation and the associated terminology.

We distinguish two types of characteristics. *Sampled Characteristics*, collected in $\mathbf{Sc}$, are attributes represented by their distribution within the blob. For example, a sampled characteristic *Age* may have categories *Child*, *Adult* and *Elder*, each associated with a certain number of individuals. By combining multiple sampled characteristics, we can jointly describe attributes such as age, level of schooling and employment status. *Traceable Characteristics*, collected in $\mathbf{Tc}$, are attributes whose values are explicitly stored as a single value for the entire blob, rather than as a distribution over categories. For instance, a traceable characteristic *Vaccination status* with values *Vaccinated* and *Unvaccinated* implies that each blob in the simulation is composed entirely of vaccinated or entirely of unvaccinated individuals.

These characteristics allow us to specify the population in more detail, e.g., elderly people who are currently vaccinated against a given disease or the number of adults who commute to work by bus. Following the previous examples, it becomes possible to track vaccination by age group in the simulation by identifying all blobs B with a *Vaccinated* status and their respective *Age* distributions. Figure 3 shows a diagram of a blob, including its characteristics and associated distributions. Appendix A.2 presents an example of the input data required to model the initial population in simulations.

As mentioned before, during the simulation, blobs in P_i are affected by the local routine Ro_{P_i} . Routines can select a specific population profile to be affected, e.g., moving only children to school, i.e., people from a particular sample attribute “Age” in the category “child” should go from P_i to P_j . This type of operation in the population B_i of a certain P uses a population template.

A population template T specifies which individuals are affected by an action and in what quantity. We represent it as $T = (T_Q, \mathbf{Tc}, \mathbf{Sc})$, which specifies i) the quantity of people to be affected, ii) an optional a set of predicates on traceable characteristics, and iii) is an optional set of predicates and distributions on sampled characteristics. When applied to a set of blobs, a template behaves as a filter that identifies the subset of individuals that satisfy all predicates and are counted towards T_Q .

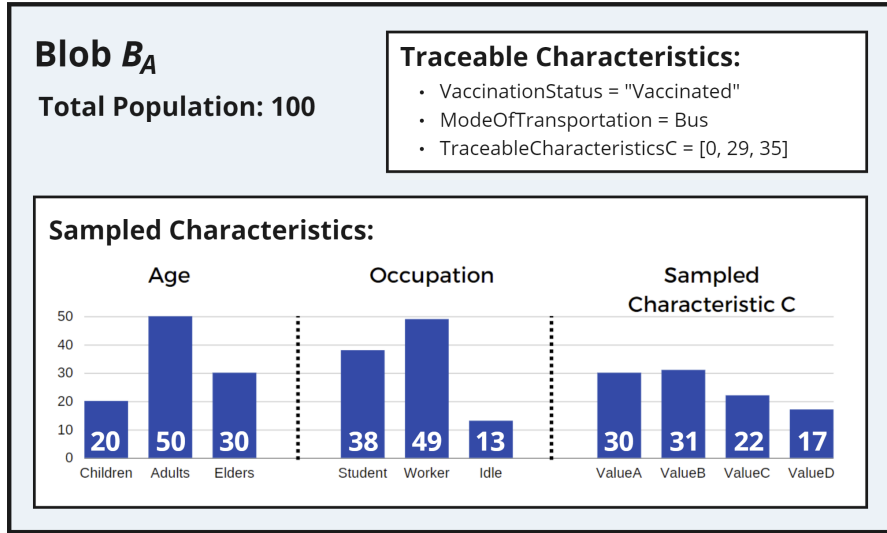


Fig. 3 Diagram of a Blob B_A containing 100 people, three traceable characteristics, and three sampled characteristics. Traceable characteristics describe the entire blob population and can have any value type. The distribution of each sampled characteristic sum should be equal to the total population of the blob.

Consider a blob B_j with two sampled characteristics: *i*) *Age*, which includes the possible values *Children*, *Adults*, and *Elders*, and *ii*) *Occupation*, with possible values *Student* and *Worker*. A template T that specifies *Age=Adults* in its collection of sampled characteristics will target only the specified group within B_j , in this case, only adults. If the template T does not include a filter for the second sampled characteristic (*Occupation*), all occupation categories in B_j are affected. The number of affected individuals in categories such as *Students* and *Workers* is determined proportionally based on their distribution within B_j . For instance, if B_j contains more students than workers, a larger proportion of students will be selected. This process continues until the total number of individuals defined in T is selected or until there are no remaining eligible individuals in B_j .

When a blob is operated using a population template T , it is possible that only a portion of its population may be affected, e.g., a blob B_i that has 75 people and only 20 are selected in T . If the operation is to move to another P , then such 20 people will create a new blob. This operation generates a split, i.e., the original blob will be separated into two smaller ones. In such an example, a template with quantity $T_Q = 20$ selects 20 people from B_i and splits it by creating a new blob B_j , which is moved to a different P . The same occurs when modifying a blob's Tcs . For example, another blob B_k has 90 *Healthy* people, and 20 are selected, using T , to be infected by a disease. The split occurs, and a *InfectionStatus* of B_l (newly created blob) is changed to *Infected*.

In addition to split operations, blobs can also be merged, combining their total populations TP . Each sampled characteristic of the resulting blob is the sum of the respective characteristics of the merged blobs. A merge can only occur between blobs

currently in the same P and with the same values in all their traceable characteristics. This restraint allows us to merge only populations that share the same traceable features. Figure 4 presents a diagram of blobs affected by two templates, resulting in splits and merges. The first scenario, depicted at the top of the figure, demonstrates the relocation of a population. The second scenario illustrates changes in traceable characteristics of a population.

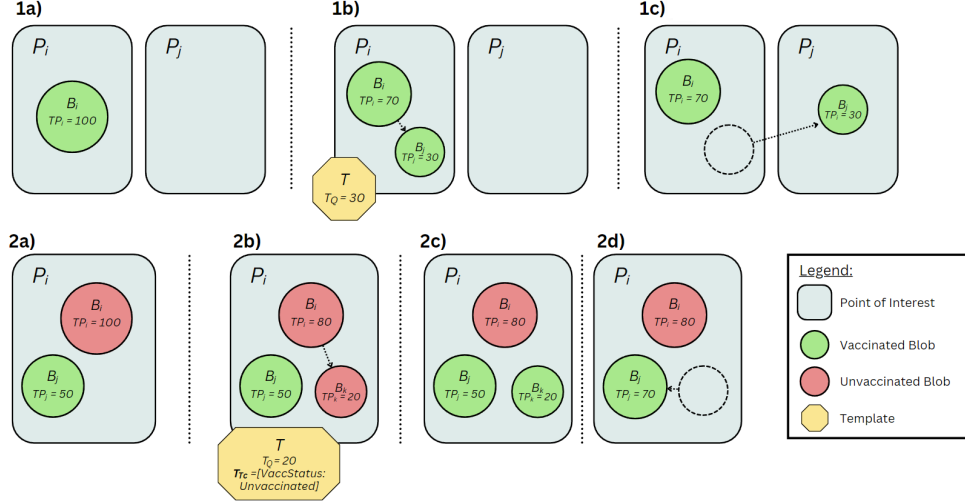


Fig. 4 Diagram presenting two scenarios of blobs being modified. The first scenario (1) (on the top) shows a population being moved to another location. In 1a, the initial blob B_i has a population of 100. In 1b, a template T selects 30 people, splitting B_i and creating B_j . In 1c, B_j is moved to P_j . The second scenario (2) shows a population's traceable characteristics changed. In 2a, there are two blobs, B_i containing 100 unvaccinated people (in red), and B_j containing 50 vaccinated people (in green). In 2b, a template T selects 20 unvaccinated people, affecting only B_i and splitting it into B_k . In 2c, the traceable characteristic $VaccStatus$ of B_k is changed to *Vaccinated*. In 2d, B_k is merged into B_j since they now share the same characteristics and are in the same P_i .

3.3 Routines and Actions

All behavioral dynamics in LodusPop are driven by routines, which associate discrete time steps with actions that operate on blobs. We consider a simulation organized into cycles of fixed length d steps. Within a cycle, time is indexed by an integer $t \in \{0, \dots, d - 1\}$, which we refer to as the cycle step.

A (local or global) routine is a finite set of pairs $R = \{(t, A)\}$, where t is a cycle step and A is an action to be executed at that step. For each point of interest P_i , the local routine is denoted Ro_i and specifies the actions that should be performed by P_i . The Global Routine GRo has the same structure but applies its actions to all points of interest in the environment.

Each action A is defined as $A = (N, T, \mathbf{V})$, where N is an identifier for the action type (e.g., MOV or CH); T is a population template as defined in Section 3.2, selecting

Table 2 Mathematical notation and descriptions of the elements defined in Section 3.2.

Element	Notation	Description
Blob	B	Represent an abstraction of the population. A B_i is defined as its total population (TP_i), a collection of traceable characteristics (Tc_i), and a collection of sampled characteristics (Sc_i)
Traceable Characteristic	Tc	Characteristic of a blob defined as an identifier (i.e., a string) and its value (not restricted by type).
Sampled Characteristic	Sc	Characteristic with a categorical distribution over the blob population. Defined as a string identifier and a collection of its possible categories, each associated with a certain number of people.
Population Template	T	Specifies which individuals are affected by an action and in what quantity, operating as a filter to select a population from a B .

the subset of a blob that will be affected; and $\mathbf{V}$ is a vector of additional values (parameters) used by the action. Table 3 summarizes the notation introduced in this section.

In the core implementation, LodusPop provides two built-in action types: *move population* (MOV), which moves a subset of a blob between points of interest; and *change traceable characteristic* (CH), which modifies the traceable characteristics of a subset of a blob.

A MOV action expects its parameter vector $\mathbf{V}$ to specify an origin and a destination point of interest. For example, the local routine of P_i may include the pair $(t = 15, A)$, where $A = (MOV, T_i, V_i)$, with $T_i = (T_Q = 20, T_{Tc} = \emptyset, T_{Sc} = \{Age = Child\})$, and V_i encoding P_i as origin and P_j as destination. At cycle step $t = 15$, this action selects 20 children from the blobs at P_i , splits them into a new blob, and moves that blob to P_j .

A CH action expects $\mathbf{V}$ to contain the name of a traceable characteristic and its new value. For instance, the local routine of P_j may include $(t = 25, A')$, with $A' = (CH, T_j, V_j)$, where $T_j = (T_Q = 20, T_{Tc} = \{Status = Unvaccinated\}, T_{Sc} = \emptyset)$, and V_j encodes the update “Status := Vaccinated”. At cycle step $t = 25$, this action selects 20 unvaccinated individuals from the blobs at P_j , splits them into a new blob, and changes the traceable characteristic Status of that blob to Vaccinated. Figure 4 illustrates both the movement and characteristic-change scenarios.

The same action representation is used by the Global Routine *GRo*. For example, a MOV or CH entry in *GRo* is executed at the specified cycle step t for all points of interest simultaneously, enabling the modeling of global phenomena such as random walks or disease spread. Appendix A.3 provides an example of the input data used to encode both local routines and the Global Routine.

New action types can be added via plugins. A plugin registers an additional identifier N together with a function that consumes actions of that type. Plugin actions retain the same structure $A = (N, T, V)$. The simulation still uses the population template T to select the affected individuals and passes T and $\mathbf{V}$ to the plugin’s function. Appendix A.4 illustrates how plugins can be used to introduce custom action types into simulations.

To achieve repeating temporal behavior, the simulation iterates over cycle steps $t = 0, \dots, d-1$ within each cycle. At the beginning of each step t , the simulator collects all actions (t, A) from *GRO* and from every local routine. For each action, it:

1. uses the template T to select the affected population from the current blobs;
2. applies the action semantics (e.g., move or characteristic change), which may split existing blobs or create new ones; and
3. performs, at the end of the step, a merge operation at each point of interest, combining blobs that share identical traceable characteristics.

Figure 5 illustrates the simulation behavior over a single cycle. Appendix A.5 presents an example of a simulation script that manages the overall control of the simulation flow. In the current implementation, actions scheduled for the same cycle step are executed sequentially in the order in which they are listed in the input, and we apply the same ordering to actions in the global routine *GRO*. Each action is treated as an atomic operation: it selects a population according to its template based on the current blob configuration, applies the corresponding modifications, and immediately updates the state before the next action is evaluated. As a consequence, the same blob (or its descendants) may be affected by multiple actions within a single cycle step, but the later actions always see the state resulting from earlier ones in that step.

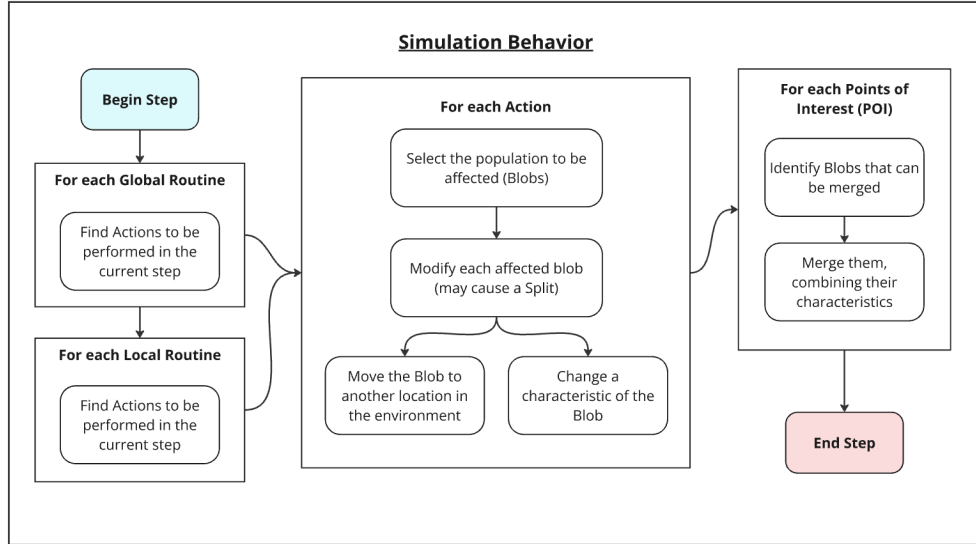


Fig. 5 Diagram of the simulation behavior, illustrating the identification and execution of actions from global and local routines. At each cycle step, all actions scheduled for that step are collected and executed sequentially in the order defined by the simulation input. Each action modifies blobs by moving them or changing their traceable characteristics and may cause splits; subsequent actions in the same step operate on this updated state. At the end of the step, blobs with identical traceable characteristics in the same point of interest are merged.

Table 3 Mathematical notation and descriptions of the elements defined in Section 3.3.

Element	Notation	Description
Local Routine	Ro	Model the repeating temporal behavior of a population B located in any given P . It is defined as $\{t, A_i\}^*$, where t is the time in the simulation that action A_i should be performed.
Global Routine	GRo	Modeling the global behavior of a simulation. Similar to a Local Routine, but it is applied to all P s in the simulation
Action	A	Represents an action to be performed in the population, either moving or changing its characteristics. A A_i is defined as an identifier for the action type ((N_i)), a population template to select the population to be affected ((T_i)), and a collection of values to be used by the action ((V_i)).

4 Experimental Results

This section presents a series of experiments designed to evaluate the key aspects and capabilities of our proposed model, LodusPop. These experiments demonstrate the model’s versatility and effectiveness in simulating population dynamics under various scenarios, highlighting its ability to manage complex interactions between the environment and the population. Each experiment highlights specific features of the model:

1. **Random walk behavior** simulates daily routines based on the population’s profile (i.e., sampled characteristics) and the proximity of points of interest. For instance, students can be modeled to attend schools during the day, while adults visit nearby pharmacies or supermarkets (Section 4.2);
2. **Large-scale events** illustrate how disruptions to daily routines impact mobility and how multiple events can compete for the available population (Section 4.3);
3. **Infectious disease simulation and vaccination scenarios** model population behavior over extended periods, illustrating how traceable characteristics evolve and how the spread of a disease impacts the population. These scenarios also enable the analysis of dynamic population attributes, demonstrating how public health policies, such as vaccinations or lockdowns, influence disease transmission and population behavior (Section 4.4).

All experiments extend the basic functionalities of LodusPop by incorporating plugins that introduce additional action types and traceable characteristics, enabling precise tracking of population evolution throughout the simulations. Additionally, these experiments demonstrate the model’s capability to represent populations as abstract blobs of varying sizes, from coarse city-wide aggregates down to blobs that represent very small groups or even single individuals. Macro-scale dynamics are evident when large groups participate in large-scale events across the environment, resulting in changes to mobility patterns (Section 4.3), while finer-grained effects appear when a small subset of the population at a specific location is affected, such as during a disease outbreak (Section 4.4). The blob sizes are dynamically adjusted

during simulations, ranging from entire neighborhood populations to single individuals. Together, these case studies validate the model’s performance and highlight its potential for practical applications in urban planning and public health.

4.1 Case-Scenario: Porto Alegre, Brazil

LodusPop, designed to work with real-world data, was tested to model the population behavior of Porto Alegre, Brazil. Following our proposed definitions for the environment (Section 3.1), we modeled each neighborhood of the city as a region (R) that contains a set of points of interest (P). Environmental data representing the boundaries of each neighborhood was obtained through a dataset made available by Porto Alegre’s city hall¹. Figure 6 presents the neighborhoods of Porto Alegre. All 94 neighborhoods of the city are included in our experiments. Four specific neighborhoods are highlighted for their additional routines, which are utilized in our large-scale experiments (Section 4.3).

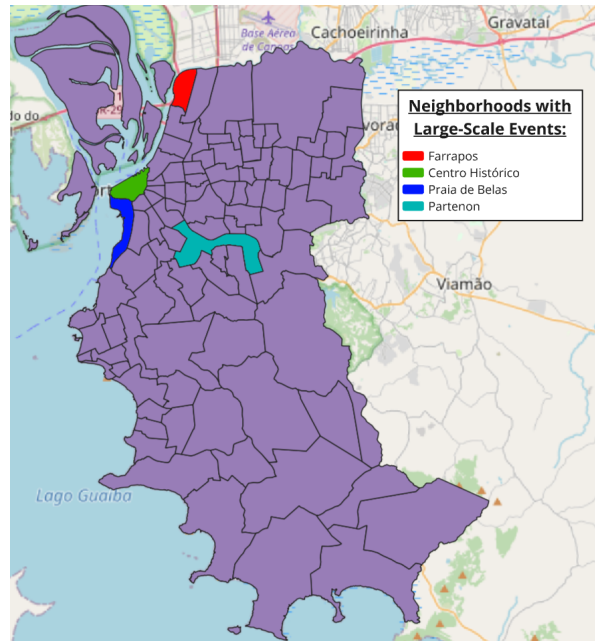


Fig. 6 The 94 neighborhoods of Porto Alegre, Brazil. All 94 neighborhoods of the city are included in all our experiments, with four specific neighborhoods highlighted for their additional routines, utilized in our large-scale experiments (Section 4.3).

We utilized the OpenStreetMap database² to extract the geographic coordinates (latitude and longitude) of buildings and amenities within Porto Alegre that could serve as points of interest for the population, such as schools and pharmacies. These

¹Neighborhood boundaries Shapefile are available at (under “Bairros (LC 12.112/16)”): <https://prefeitura.poa.br/carta-de-servicos/mapas-digitais-da-smamus>

²OpenStreetMaps is available at: <https://www.openstreetmap.org/>

points of interest were distributed across the city’s neighborhoods. Each region in the model includes the following *Ps*: *home*, *work*, *school*, *marketplace*, *pharmacy*, and *gas_station*. Additionally, some regions may feature other *Ps*, such as *shopping_malls*, *hospitals*, *universities* or *stadiums*. The modeled environment comprises 94 *Rs* and a total of 504 *Ps*. For the experiments described in this paper, each region contains at most one *P* of each type, such as one *home* and one *pharmacy* per neighborhood. This setup was dictated by the available data, which provided information on population distribution at the neighborhood level but did not specify finer-grained details within neighborhoods. However, this is not a limitation of the proposed model. When more granular data is available, simulation designers can enhance the environment by adding multiple home *Ps* within a single region and allocating neighborhood populations accordingly. Appendix A.1 provides an example of the syntax needed to inform data for modeling an environment in our simulations.

To model the population in our simulations, we collect data from governmental sources, including the most recent completed census at the time of this publication³. Census data provided citywide information on the total population, the number of economically active individuals (i.e., workers), the number of students, and the age distribution of the population. However, this information lacked specificity at the neighborhood level. To address this, additional data was gathered from city administration surveys, which included neighborhood-level details on total population and age distribution⁴⁵. This information was used to define sampled characteristics for the population, with the following identifiers and categories:

- *Age*: Child, Young, Adult, and Elder;
- *Occupation*: Student, Worker, and Other.

Porto Alegre’s total population is 1,409,351. Since *occupation* data was only available at the city level, all neighborhoods were assigned the same distribution: *students* comprise approximately 21.575% of the population, *workers* 54.706%, and *other* 23.718%. The *age* distribution was mapped using the age pyramid, with each category representing a specific range: *child* (0–14 years), *young* (15–24 years), *adult* (25–59 years), and *elder* (60+ years). Each neighborhood has its own unique *age* distribution, while the overall population consists of approximately 18.724% *children*, 15.706% *young*, 50.507% *adults*, and 15.061% *elders*.

For each neighborhood, a blob was initialized at the corresponding *home P*, with its total population determined based on the available neighborhood data and its sampled characteristics determined according to the previously described *age* and *occupation* distributions. Each blob was also assigned a traceable characteristic, *region of origin*, which indicates where the population resides and begins within the simulation. This traceable characteristic enables the identification of a blob’s origin throughout the simulation and ensures that the population returns to their respective residences after completing daily activities. Moreover, blobs originating from different regions

³Census Conducted by the Brazilian Institute of Geography and Statistics (IBGE). The data is available at: <https://cidades.ibge.gov.br/brasil/rs/porto-alegre/panorama>

⁴Surveys conducted by the Observatory of the City of Porto Alegre (ObservaPOA). Data is available at: <http://www.observapoa.com.br/>

⁵Surveys conducted by the Empresa Pública de Tecnologia da Informação e Comunicação da Prefeitura de Porto Alegre (PROCEMPA). Data is available at: <https://portoalegremanalise.procempa.com.br/>

cannot be merged, as their traceable characteristics remain distinct, preserving information about their place of origin. Appendix A.2 provides an example of the input data necessary for modeling an initial population in our simulations. For all experiments, a *cycle* was defined as 24 *cycle steps*, representing a full day and its hours, respectively. The following sections present results obtained from various simulation scenarios, illustrating the model’s capacity to capture population dynamics effectively.

4.2 Case Study 1: Random Walk

Gonzalez et al. [31] provide valuable insights into the dynamics of urban populations, which serve as a foundation for our abstracted urban mobility model. They highlight two significant observations: first, individuals tend to visit a select few locations repeatedly, and second, there is a certain level of randomness in their daily movements, akin to a Lévy flight random walk. A Lévy flight is a random walk pattern characterized by jumps of varying lengths, following a Lévy heavy-tailed probability distribution. In practical terms, this means that the Lévy flight predominantly consists of short jumps, with fewer long jumps.

To simulate this behavior, we have implemented a Lévy plugin, which introduces a new action type called the *levy walk*. Appendix A.4 presents an example of how new action types can be incorporated into our simulations using plugins. This action involves sampling distances from a Lévy distribution and moving a certain portion of the population from a certain origin P_i to a destination P_j corresponding to the sampled distance. For each P , we calculate a histogram of distances to other P s, grouping them into bins based on a specified distance grouping value. Figure 7 illustrates the histogram of distances between P s, with each bin representing a range of 500 meters.

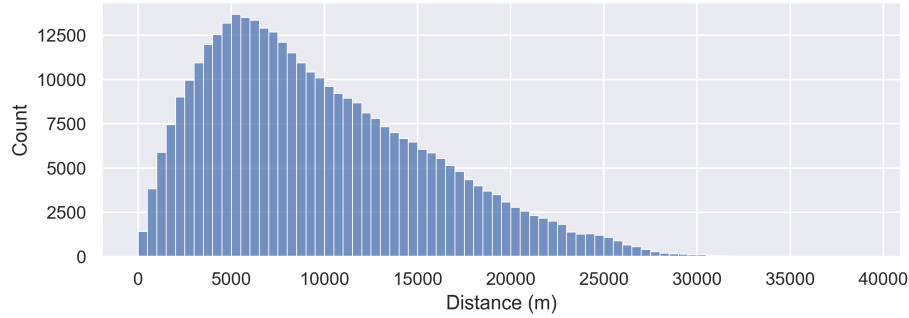


Fig. 7 Histogram of distances between points of interest (P s). Each bin represents a range of 500 meters.

When the *levy walk* action is triggered, the population at the origin P_i is uniformly divided into packets to be moved. Upon sampling a distance, the *levy walk* action randomly assigns a population packet to one of the P s within that distance group. Since the Lévy distribution is heavy-tailed (see Figure 8), it is possible to sample a distance that exceeds the farthest P , even if it is not very probable. In such cases,

another distance is sampled from the distribution. Similarly, if there are no available P s within the sampled distance group, a new distance will be sampled.

A *levy walk* action A_i incorporates additional parameters $\mathbf{V}_i$ (Section 3.3) to enhance its functionality. These parameters include the scale parameter (*levy_scale*), which governs the Lévy distribution; the movement probability parameter (*levy_prob*), which determines the likelihood of a population packet being moved; the packet size parameter (*levy_pkt*), representing the group size that undergoes the random walk; and an optional label parameter (*levy_label*), enabling the selection of specific types of P s as potential destinations. Additionally, a *levy walk* action A_i requires a population template T_i to specify the desired population profile being affected. These parameters enable the simulation of random walk behavior across multiple scales: spatial scales, by adjusting the *levy_scale* parameter to match the simulated environment, and population scales, by modifying *levy_pkt*, which allows for the movement of large populations or individual entities (i.e., a blob with a total population of 1).

In our experiments, we design a global routine (*GRO*) that utilizes *levy walk* actions to disperse the working and student populations across the city throughout the day. Actions targeting workers relocate the population from a “home” P_h to a “work” P_w , utilizing a population template with criteria such as *Occupation* = *Worker* and *Age* = [*Young, Adult, Elder*]. These actions occur at each cycle step, with a higher movement probability (*levy_prob*) during the morning and afternoon (specifically, steps 7 through 18). Actions targeting students relocate the population from a “home” P_h to a “school” P_s , using a population template with criteria such as *Occupation* = *Student* and *Age* = [*Child, Young, Adult*]. These actions are executed only at specific cycle steps. Furthermore, the simulation routine includes *move population* actions in all “work” and “school” P s, returning a portion of their current population to their respective “home” P of each blob they contain. In both cases, the *region of origin* traceable characteristic of each blob allows us to identify the “home” P s where students and workers originate and return at the end of the day.

Subsequently, we conducted experiments by varying the *levy_scale* parameter in all *levy walk* actions to assess the population movements throughout the simulation. We define “displacement” as the movement of an individual from one P_i to another P_j , as part of the levy plugin. In the case of a blob B_i with a total population $TP_i = 250$, each movement between points of interest accounts for 250 displacements. All simulations maintain a population packet size of *levy_pkt* = 250, and the P s are grouped based on the distance to an origin P , with intervals of 500 meters, as depicted in Figure 7.

The experimental results are illustrated in Figure 8. All simulations consider a total population of 1,409,351. The x-axis represents the distance traveled during each displacement, grouped into 500-meter intervals. The y-axis indicates the total number of displacements that occurred at each levy configuration. The graph demonstrates that different values of the *levy_scale* parameter produce distinct distributions, where higher values yield more widely dispersed distributions of displacements, as observed in a simulation where *levy_scale* = 10.

For the remaining experiments in this work, we adopt a scale parameter of *levy_scale* = 2 (represented by the green line in Figure 8). This particular value was determined empirically, as approximately 50% of the displacements observed

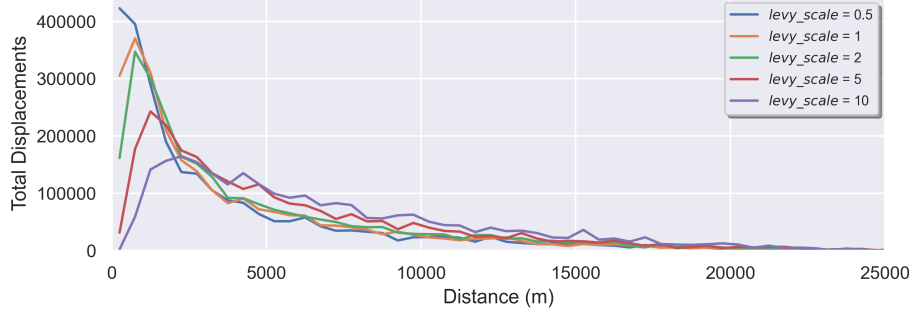


Fig. 8 Population displacement experiments. The x-axis represents the distance traveled during a displacement and is grouped in ranges of 500 meters. The y-axis represents the total number of displacements that traveled a certain distance. All simulations consider a total population of 1,409,351; a population packet size of $levy_pkt = 250$, which represents the group size that undergoes the random walk; and P s grouped by distance in ranges of 500 meters, as presented in Figure 7.

in simulations using this parameter occurred within a 2km radius of the origin P . Such displacements align with typical daily movements within a single neighborhood or between neighboring neighborhoods. However, this value can be adjusted to suit different environments, depending on the needs of the simulation designers.

Furthermore, we introduce an additional parameter, named *Restriction*, for the *levy walk* action to model movement restrictions. This parameter acts as a factor that reduces the total population being moved by each *levy walk* action, e.g., an $Restriction = 0.25$ would cause a *levy walk* action to move only $\approx 75\%$ of the population it would without the restriction. It allows for the simulation of varying population flows on different days of the week (e.g., weekends with different flow patterns than weekdays) or the simulation of population lockdown during a pandemic scenario. Figure 9 illustrates the results obtained by conducting a baseline experiment (without any restriction) and three other experiments involving different levels of movement restrictions. As previously mentioned, all simulations consider a scale parameter of $levy_scale = 2$.

4.3 Case Study 2: Large-Scale Events

We also investigate the ability to simulate occasional events that disrupt the population’s daily routines, such as sports matches or musical concerts that occur on a weekly or monthly basis. To evaluate the impact of these large-scale events on mobility patterns, we compare their effects against a baseline scenario with no events. Furthermore, we examine how multiple concurrent events compete for the available population, highlighting the dynamics of population distribution.

To simulate these events, we implemented an additional plugin, which introduces a new action type called *gather population*. This action involves gathering the population from nearby P s and relocating them to a designated destination P_d . A *gather population* action A_j considers a *gather_quantity* additional parameter (in $\mathbf{V}_j$) to describe

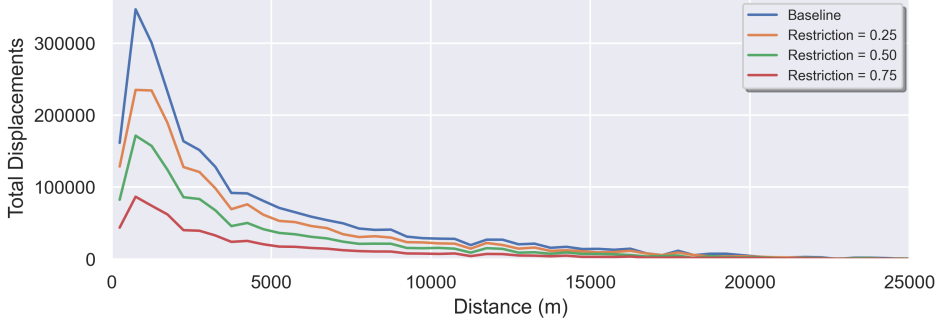


Fig. 9 Simulation of Movement Restriction with the *levy walk* action. All simulations consider a scale parameter of $levy_scale = 2$. The *Restriction* parameter acts as a factor that reduces the total population being moved by each *levy walk* action.

the total population to be relocated towards the destination P_d . The population quantity to be moved from each nearby P is determined by the population available at the origin P and the distance to the destination. Once the population quantity that should be moved from each origin P is determined, the *gather population* action is further broken down into multiple *move population* actions. Each of these *move population* actions is responsible for transferring the population from an origin P to the destination P_d , ensuring that each origin P contributes to the movement.

For these experiments, we consider the routines of workers and students as a baseline simulation, utilizing a scale parameter of $levy_scale = 2$ as described in Section 4.2. Furthermore, we incorporate four large-scale events into the simulation, each occurring at a specific cycle step. These events are characterized by a *gather population* action, which requests the movement of 500,000 individuals to a designated destination P . The details of the events are specified as follows, considering the regions illustrated in Figure 6:

- Event (A): A sports event taking place at the Farrapos region stadium;
- Event (B): A sports event happening at the Praia de Belas region stadium;
- Event (C): A commercial event held at the marketplace in the Centro Histórico region; and
- Event (D): A cultural event occurring at our University in the Partenon region.

All events are scheduled to occur at cycle step 19, i.e., at 7:00 PM. Given that the city’s total population is 1,409,351, and the events require a total of 2,000,000 individuals, the events will compete for the available population if they all occur simultaneously in a single simulation. In addition, we model a scenario where some large-scale events take place at different cycle steps. In this alternate scenario, event C occurs at cycle step 9 (i.e., 9:00 AM), while event D occurs at cycle step 14 (i.e., 2:00 PM).

Table 4 provides an overview of the simulation results, including the total population displacements, the average displacement per city inhabitant, and the proportion

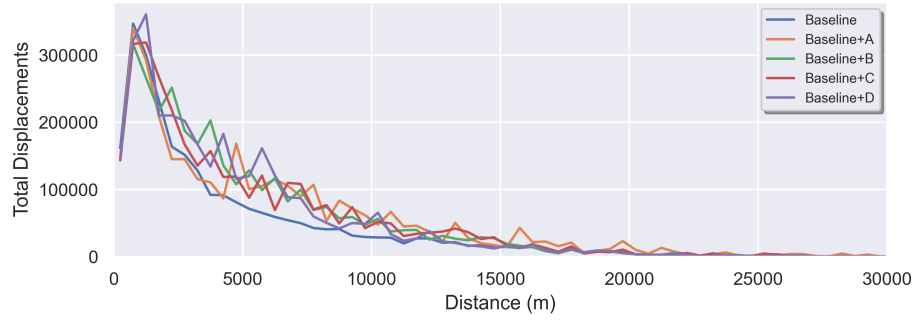
of total displacements compared to the baseline simulation. Baseline means the “normal life” of the city, and the different scenarios in the table indicate the population displacements as a function of the existence of the events. For instance, the “Baseline+AB” scenario considers the Baseline in addition to Event A and Event B. In the two last lines of Table 4, we show the results of the Baseline with the existence of the four large-scale events occurring at the same time and with some occurring at different cycle steps.

Table 4 Results of the large-scale event simulations. Data includes the total population displacements, the average displacement per inhabitant of the city, and a proportion of total displacements compared to the baseline simulation. The “Baseline+AB” scenario, for example, considers the Baseline in addition to Event A and Event B.

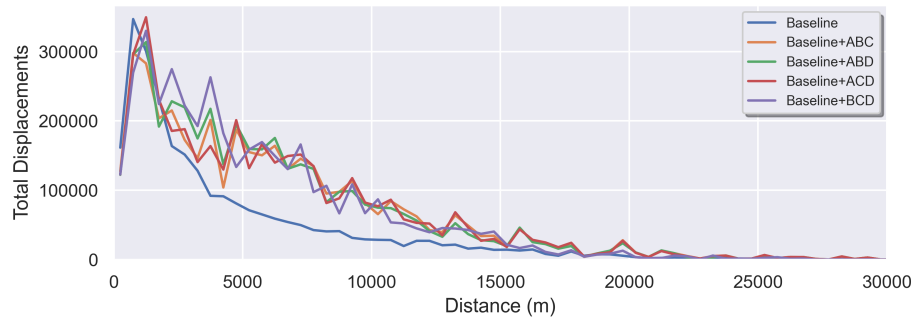
Experiment Name	Total Displacements	Average Displacements per inhabitant	Increase over Baseline
Baseline	2,570,658	1.8240	—
Baseline+A	3,290,469	2.3347	28.00%
Baseline+B	3,300,782	2.3421	28.40%
Baseline+C	3,289,002	2.3337	27.94%
Baseline+D	3,281,102	2.3281	27.64%
Baseline+AB	3,786,856	2.6870	47.31%
Baseline+AC	3,748,417	2.6597	45.82%
Baseline+AD	3,804,017	2.6991	47.98%
Baseline+BC	3,738,497	2.6526	45.43%
Baseline+BD	3,839,126	2.7240	49.34%
Baseline+CD	3,826,094	2.7148	48.84%
Baseline+ABC	4,042,413	2.8683	57.25%
Baseline+ABD	4,109,929	2.9162	59.88%
Baseline+ACD	4,069,223	2.8873	58.29%
Baseline+BCD	4,081,563	2.8961	58.78%
Baseline+ABCD	4,235,926	3.0056	64.78%
Baseline+ABCD+DiffStep	4,638,329	3.2911	80.43%

Figure 10 presents the results of the simulation of large-scale events. Figure 10a visualizes the results of five simulations: the Baseline simulation (no events), a simulation with only Event A, a simulation with only Event B, only Event C, and only Event D. As indicated by the data in Table 4, simulations where only a single large-scale event occurs yield a similar number of total displacements. However, the different locations where each event occurred (i.e., different destination P s) and the available population nearby cause each simulation to have other total displacement counts for different distances. For example, the “Baseline+A” simulation (shown in orange in Figure 10a) presents an increased total displacement count near the 20,000m distance.

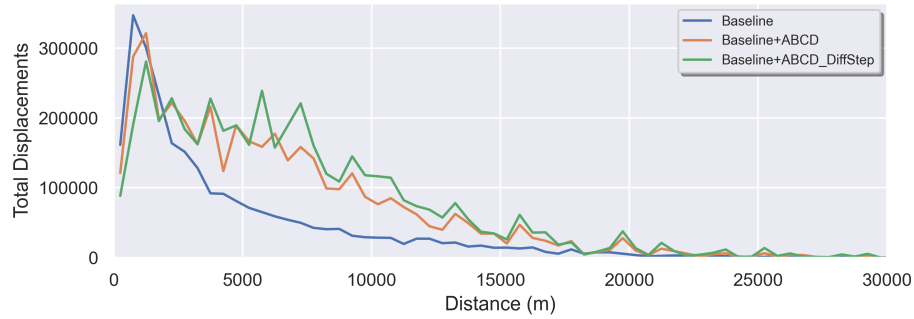
Figure 10b illustrates the results of five simulations, starting with the baseline simulation and followed by simulations with three events occurring at the same cycle step (i.e., only one event missing in each). As expected, and as indicated by the data in Table 4, the inclusion of additional events led to an increase in total displacements compared to the baseline simulation. However, it is important to note



(a) Experiments include the Baseline scenario (no events), a scenario with only Event A, only Event B, only Event C, and only Event D.



(b) Experiments include the Baseline scenario and four other scenarios with three events occurring at the same cycle step (i.e., only one event missing in each).



(c) Experiments include the Baseline scenario, the Baseline in addition to all four events occurring at the same cycle step, and the Baseline in addition to all four events with some occurring at different cycle steps (DiffStep).

Fig. 10 Simulation of large-scale events.

that although each event requests the movement of 500,000 individuals, the total displacements of three events do not increase by 1,500,000 compared to the baseline simulation. This is because individuals can move directly from their respective origin P s (such as work or school) to the event P s. Thus, workers can travel from their “work” P s to the event directly, and students can travel from their “school” P s to the event without the need to return to their “homes” before going to the events. Moreover, Events A, B, and C separately contribute to $\approx 730,000$ additional displacements to the baseline simulation, as shown in “Baseline+A”, “Baseline+B”, and “Baseline+C” in Table 4, respectively. In simulations where the three events occur simultaneously (i.e., “Baseline+ABC”), the events will compete for the available population in nearby neighborhoods, resulting in a reduced total population attending each event. Such result is presented in the last column of Table 4, where the number of displacements increased over the baseline is 57.25% for simulation “Baseline+ABC”. This is a lower amount than the sum of increased displacements of simulations “Baseline+A”, “Baseline+B”, and “Baseline+C”, which were 28.00%, 28.40%, and 27.94%, respectively.

Figure 10c presents the results of three simulations: the baseline simulation, a simulation with all four events occurring simultaneously, and a simulation with some events taking place at different cycle steps. As demonstrated by the data in Table 4, the simulation where events occurred at different cycle steps exhibited a significantly higher number of displacements than the simulation with all four events happening simultaneously. This increase in the number of displacements can be attributed to two factors. Firstly, there is reduced competition for the available population when events are staggered. Secondly, individuals have the opportunity to attend multiple events throughout the day, increasing their individual displacements.

4.4 Case Study 3: Epidemic Contagion and Vaccination

Next, we explore our model’s ability to simulate behaviors observed over extended temporal scales, focusing on their long-term effects on the population. Specifically, we model the spread of an infectious disease as the population moves throughout the city, analyzing its progression over multiple simulation cycles, which represent consecutive days. Additionally, we evaluate the impact of public policies, such as vaccination campaigns and population lockdowns, on mitigating the disease’s spread.

To achieve this, we have implemented an additional plugin that introduces an *infection* action type. This action is based on the SIR model [43], which classifies individuals into one of three states: Susceptible, Infected, or Removed. The “Removed” state includes individuals who can no longer contribute to further contagion, either due to acquiring immunity (recovered) or death. The numerical dynamics of the SIR model are defined by Equations 1, 2, and 3.

$$\frac{dS}{dt} = -\frac{\beta IS}{N}, \quad (1)$$

$$\frac{dI}{dt} = \frac{\beta IS}{N} - \gamma I, \quad (2)$$

$$\frac{dR}{dt} = \gamma I, \quad (3)$$

where, $\frac{dS}{dt}$, $\frac{dI}{dt}$ and $\frac{dR}{dt}$ represent the rate of change over time for the susceptible, infected, and recovered populations, respectively, in a total population of N individuals. The parameters β and γ represent the infection and removal rates, respectively. The basic reproduction ratio, denoted as R_0 , is defined as $R_0 = \frac{\beta}{\gamma}$. It is important to note that the total population N remains constant in the SIR model, maintaining equality throughout the simulation.

To monitor the progression of the infection within the population, we introduced a traceable characteristic called *SIR Status* for all blobs in the simulations. This characteristic can take one of three possible values: “Susceptible”, “Infected”, or “Removed”. As outlined in Section 3.2, each blob in the simulation consists of individuals who share the same *SIR Status* value. However, blobs with different *SIR Status* values can coexist within the same P , allowing for the simultaneous representation of susceptible, infected, and removed individuals at a given location.

In the context of epidemic simulations, we utilize the routines of workers and students as the baseline, employing a scale parameter of *levy_scale* = 2 as described in Section 4.2. Additionally, we incorporate the *infection* action into the global routine *GRO*, which occurs at every cycle step. When triggered, the *infection* action identifies the current population with each SIR status in a given P and applies Equations 1, 2, and 3 based on the β and γ parameters. We conducted experiments considering different values for the basic reproduction ratio (R_0), as this value can vary across different countries and cities. We specifically selected R_0 values of 2.6, 3.07, and 6.0 based on research conducted on Brazil’s COVID-19 data [44–46]⁶. For all experiments, we set $\gamma = \frac{1}{15}$, indicating that individuals transition from the “Infected” to the “Removed” state over a 15-day period. The values of β are defined as 0.1733, 0.204, and 0.4, respectively, to correspond with the R_0 values of 2.6, 3.07, and 6.0.

As part of the simulation setup, we empirically determined that each region of the environment begins with 50 randomly selected individuals classified as “Infected,” 0 individuals classified as “Removed,” and the remaining population classified as “Susceptible.” This equates to a total of 4,700 individuals initially categorized as “Infected” across all regions. The simulations were run for a period of 500 cycles, where each cycle consists of 24 cycle steps representing days and hours of the day, respectively.

Figure 11 illustrates the results of the simulations conducted with different basic reproduction ratios (R_0). The x-axis denotes the simulation cycle (i.e., day), while the y-axis shows the cumulative number of infected individuals up to that point, including those who transitioned from the “Susceptible” to the “Infected” state, as well as the initial infected population (4700). As depicted, higher values of R_0 directly influence the speed of infection spread and the cumulative number of infected individuals throughout the simulation.

For the remaining experiments, we will consider a baseline value of $R_0 = 2.6$ for the simulations. Figure 12 displays the total population amount for each possible value of the “SIR Status” traceable characteristic over time in a simulation with $R_0 = 2.6$.

⁶Additional research on R_0 on Brazilian state capitals available [HERE](#)

Comparison of Cumulative Infected Population

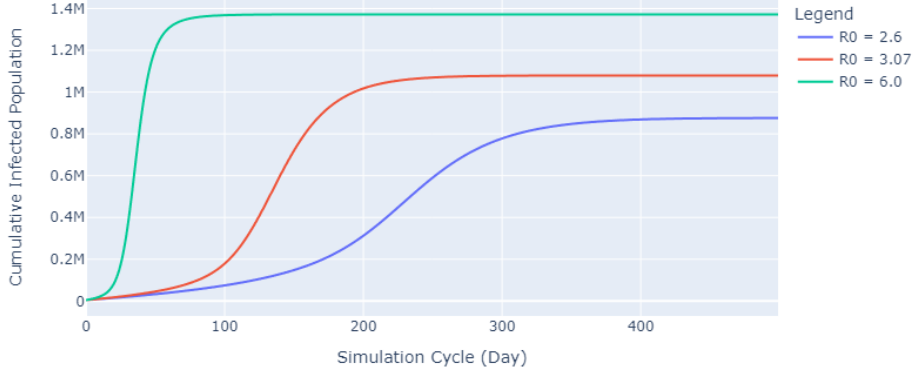


Fig. 11 Simulations conducted with different basic reproduction ratios (R_0). The x-axis represents the simulation cycle (i.e., day), while the y-axis represents the cumulative infected population up to that point in the simulation.

It can be observed that despite reaching a cumulative infected population of 875,962 people (as shown in Figure 11), the peak of the infection occurred on day 247, with a total of 81,528 infected individuals at that point.

Next, we extend the simulation to include additional behaviors related to epidemic contagion, specifically population vaccination as a measure to mitigate the spread of the disease. To facilitate this, we introduce a traceable characteristic called *vaccine level* to all blobs in the simulation. This property is represented by an integer value. Additionally, we implement a new action called *vaccinate*. When this action is triggered, a portion of the population is moved to a nearby “pharmacy” P , which is available in all regions of the environment. At the pharmacies, the population receives a dose of the vaccine, resulting in an increase of their *vaccine level* by 1. This signifies that they have been vaccinated and their resistance to the transmission of the virus has improved. Subsequently, the newly vaccinated population returns to their respective P s, such as workers returning to their “work” P .

In our simulations, we explore various vaccination configurations by considering different dose counts, which represent the number of doses available to the population, as well as the effectiveness of each dose. Table 5 summarizes the vaccination settings used in our experiments, where the Dose Count ranges from 1 to 3. The Effectiveness parameter indicates the efficacy of each dose in mitigating the spread of the infection.

All blobs in simulations with vaccinations contain a *vaccine level* corresponding to an Effectiveness value in Table 5. Whenever a blob with *SIR Status* = *Susceptible* gets infected, the amount of people that should be infected is defined by Equation 2. The Effectiveness works by multiplying the result of Equation 2 by $1 - Effectiveness$, i.e., an *Effectiveness* = 0.4 reduces the amount of people changing from “Susceptible” to “Infected” by 40%. Starting Day indicates the simulation cycle at which each dose

SIR Population Status - $R_0 = 2.6$

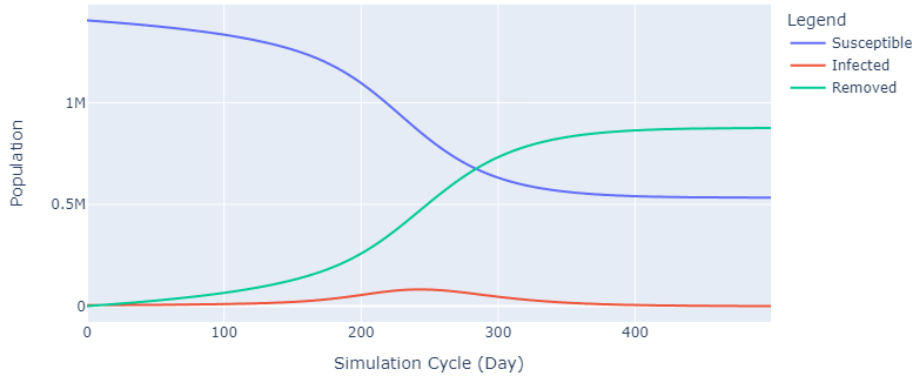


Fig. 12 Population count over time for each “SIR Status” traceable property value in a simulation with $R_0 = 2.6$

begins to be administered to the population. It is important to note that the second and third doses, if available, can only be administered to individuals who have already received the previous dose. Configurations A through C represent increasing effectiveness as more doses are administered, while Configuration D represents an ideal vaccine with 100% effectiveness in preventing infection.

Building upon the parameters defined in previous experiments, all vaccination experiments utilize the routines of workers and students as the baseline, employing a scale parameter of $levy_scale = 2$ and an *infect* action with $R_0 = 2.6$. The global routine *GRO* also includes the *vaccinate* action, performed at selected cycle steps during the morning and afternoon of each day. To determine the number of individuals to be moved to “pharmacy” *Ps* on each simulation day, we gathered data on the daily number of COVID-19 vaccines administered in Porto Alegre⁷. This data is then proportionally distributed among the regions in the environment based on their respective populations.

Figure 13 showcases the results of the vaccination simulations. Despite the “VaccinationA” configuration having an effectiveness of only 0.4, a significant impact on the cumulative infected population can be observed compared to the baseline. The baseline simulation resulted in a total of 875,962 infected individuals, whereas the “VaccinationA” configuration achieved a notably lower count of 440,569 infected individuals. The incorporation of additional doses in the “VaccinationB” and “VaccinationC” configurations further decreased the cumulative infected population, resulting in 360,607 and 355,124 infected individuals, respectively. Although “VaccinationC” contains an effectiveness of 1.0, this efficacy is only realized when the population undergoes the third dosage, initiated on day 250 of the simulation. Consequently, “VaccinationB”

⁷Dashboard of Brazilian vaccination data available [HERE](#) (text in Brazilian Portuguese)

Table 5 Configuration of vaccines for experiments. Dose Count represents the number of doses available for the population. Effectiveness represents how effective each dose is in reducing the spread of the infection. Starting Day represents the day (i.e., simulation cycle) when each dose starts being administered to the population. The second and third doses can only be applied to people who already received the previous dose. While VaccinationA, B, and C state for the doses, VaccinationD simulates a scenario where the effectiveness of the vaccine is 100%.

Vaccination Config	Dose Count	Effectiveness	Starting Day
VaccinationA	1	[0.4]	[50]
VaccinationB	2	[0.4, 0.7]	[50, 150]
VaccinationC	3	[0.4, 0.7, 1.0]	[50, 150, 250]
VaccinationD	1	[1.0]	[50]

and “VaccinationC” are very similar until day 250, at which point the third dosage of “VaccinationC” begins to exert its influence, resulting in a small difference in the cumulative infected population of both simulations. Notably, the “VaccinationD” configuration, representing an ideal vaccine, demonstrates the lowest number of infections, with only 170,978 individuals affected.

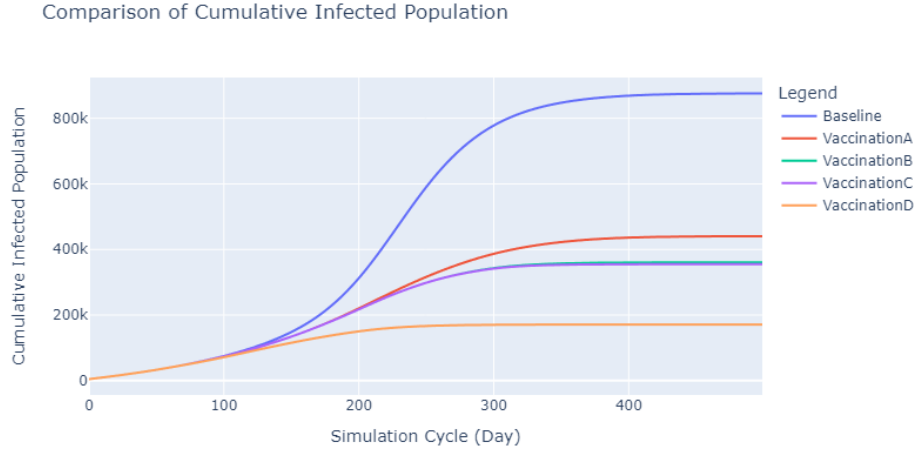


Fig. 13 Simulations of different vaccination configurations, as described in Table 5. All simulations consider a basic reproduction rate of $R_0 = 2.6$.

Finally, we explore the incorporation of movement restrictions in our epidemic contagion experiments to simulate lockdown behavior. The movement restriction is described in Section 4.2 and illustrated in Figure 9. These experiments encompass a combination of three movement restriction values (no restriction, 0.5, and 0.75)

that represent percentages of the population who do not leave their living neighborhoods, along with two vaccination scenarios: no vaccination; or “VaccinationC”, which includes three doses. We chose to compare the “VaccinationC” configuration rather than “VaccinationD” because “VaccinationC” exhibits a more realistic behavior. In “VaccinationC”, the effectiveness of the vaccine increases with the number of doses a person receives. In contrast, “VaccinationD” represents an idealized vaccine with a fixed effectiveness of 1.0 achieved in a single dose.

Figure 14 illustrates the results obtained from our simulations. The imposition of movement restrictions reduces the cumulative infected population compared to the baseline scenario (no restriction and no vaccination). However, including population vaccination (experiment “VaccC”) exhibits more significant effectiveness in reducing the infected population compared to simulations with only movement restrictions. Moreover, the combination of movement restriction and vaccination demonstrates the most favorable outcomes, with the simulation containing a movement restriction of 0.75 and vaccination resulting in only 140,941 infected individuals.

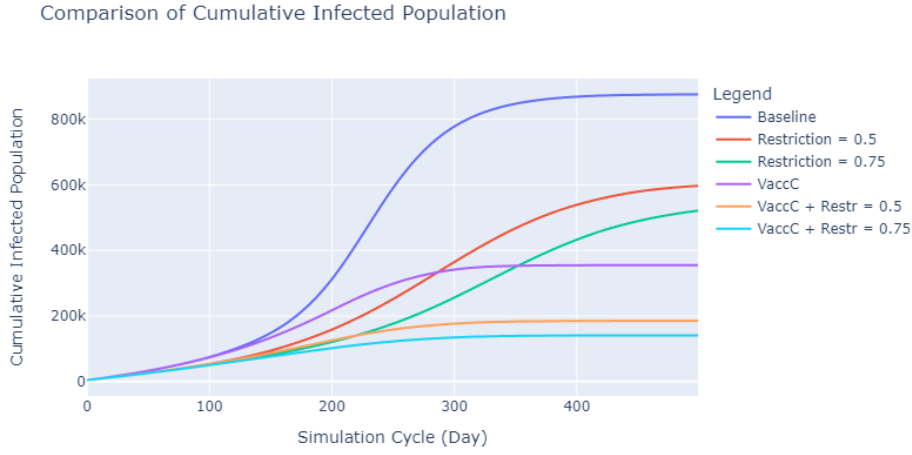


Fig. 14 Simulations of epidemic contagion. Experiments include a combination of three movement restriction values (no restriction, 0.5 and 0.75) to represent a lockdown and two vaccination scenarios (no vaccination or “VaccinationC”). All simulations consider a basic reproduction rate of $R_0 = 2.6$.

4.5 Quantitative Performance Evaluation

To complement the qualitative case studies, we present a quantitative evaluation of LodusPop’s computational efficiency. All benchmarks use the Porto Alegre city model described in Section 4.1 (94 regions, 504 points of interest, and a total population of 1,409,351). Each configuration was executed five times, and average values are reported.

Table 6 reports quantitative information for each *Case Study* and *Configuration*, as follows: (i) *Max Blob Count*, the maximum number of blobs observed during the run; (ii) *Number of Cycles*, where a cycle corresponds to one simulated day in our setup, with 24 cycle steps; (iii) *Avg Sim Time (s)*, the average total real-time required to complete the run; (iv) *Time Per Cycle (s)*, the average runtime per simulated day; (v) *MemProfile Avg Peak (MiB)*, the average peak resident memory of the Python process⁸; and (vi) *TraceMalloc Avg Peak (MiB)*, the average peak traced Python allocations⁹. We interpret *Max Blob Count* as an indicator of simulator workload: more frequent splits increase the number of population units to process and reduce merge opportunities, typically raising both runtime and memory consumption.

Using the random walk configuration of *levy_scale* = 2 (Section 4.2) as the baseline simulation for the remaining case studies and configurations, the simulator completes a full run in 5.211s on average, with *MemProfile Avg Peak* of 234.9 MiB, and a *TraceMalloc Avg Peak* of 58.565 MiB. Introducing movement restrictions (Section 4.2) slightly reduces *Time Per Cycle* (down to 4.795s at *Restriction* = 0.75), while peak memory remains essentially unchanged. This indicates that reduced mobility mainly decreases the number of executed move operations, without substantially changing blob splitting, as expected.

Large-scale events (Section 4.3) increase runtime because they introduce additional gathering actions that relocate populations across distant locations. *Time Per Cycle* rises from 6.144s (average simulations with one event) to 12.172s (simulation with four simultaneous events). These scenarios tend to produce fewer blobs, as populations aggregate around event locations, yet *MemProfile Avg Peak* and *TraceMalloc Avg Peak* remain close to the baseline (≈ 235 MiB and ≈ 59 MiB, respectively). This indicates that the dominant cost comes from executing the large-scale gathering operations by pulling individuals from multiple regions and points of interest, rather than from an increase in blob count.

Epidemic contagion and vaccination simulations (Section 4.4) are more demanding because additional traceable characteristics (e.g., SIR status and vaccine level) increase population heterogeneity, causing more frequent splits and reducing merge opportunities. In our model, blobs can only merge when their traceable attributes match; as the simulation progresses, individuals become distributed across multiple states (susceptible, infected, removed) and vaccination levels, so blobs with different traceable characteristics cannot be merged back together. For infection-only experiments, 500-cycle runs require 9.899–11.654s per cycle, and *MemProfile Avg Peak* increases to roughly 1.72–1.80 GiB. Vaccination scenarios further intensify splitting: VaccinationC reaches a *Max Blob Count* of 23,355 and requires 33.455s per cycle, with *MemProfile Avg Peak* of 2.86 GiB. Overall, *Time Per Cycle* closely follows increases in *Max Blob Count*, reinforcing that split and merge behaviors are key components of computational cost.

These measurements reflect the simulator execution on a single machine and are therefore hardware-dependent. The results are intended as a quantitative evaluation of the presented case studies, providing direct support for the efficiency and scalability

⁸Calculated with Memory Profiler v0.61.0: <https://pypi.org/project/memory-profiler/>

⁹Calculated with TraceMalloc: <https://docs.python.org/3/library/tracemalloc.html>

claims within their scope, rather than as a head-to-head comparison with other simulation frameworks. All experiments were run on a machine with an AMD Ryzen 5 5600X CPU, an NVIDIA GeForce RTX 2060 GPU, 16GB DDR4 2666MHz (CL19) RAM, and a KINGSTON SNV3S1000G SSD, using Python v3.13.6 on Microsoft Windows 11 Education (Version 10.0.262000).

Table 6 Quantitative performance evaluation of LodusPop. We use the Random Walk mobility configuration (Lévy-flight simulation with $levy_scale = 2$; Section 4.2) as the baseline reference for the remaining case studies and configurations. For each case study and configuration, we report the maximum blob count observed during execution, the number of simulated cycles (days), the average total simulation time over five runs, the corresponding time per cycle, and peak memory usage measured via MemProfile (process RSS) and TraceMalloc (Python allocation tracing).

Case Study	Configuration	Max Blob Count	Number of Cycles	Avg Sim Time (s)	Time Per Cycle (s)	MemProfile Avg Peak (MiB)	TraceMalloc Avg Peak (MiB)
Baseline (Random Walk) Case Study 1	$levy_scale = 2$	2857	1	5.211	5.211	234.912	58.565
Movement Restriction (Fig 9) Case Study 1	Restriction = 0.25	2871	1	5.074	5.074	234.957	58.577
	Restriction = 0.50	2864	1	4.958	4.958	234.935	58.527
	Restriction = 0.75	2883	1	4.795	4.795	235.234	58.516
Large-Scale Events (Fig 10 and Table 4) Case Study 2	Average of 1 Event	2671	1	6.144	6.144	235.355	58.767
	Average of 2 Events	2651	1	7.986	7.986	235.297	58.892
	Average of 3 Events	2651	1	9.910	9.910	235.530	59.061
	4 Events	2651	1	12.172	12.172	235.825	59.217
	4 Events at Different Times	2079	1	11.153	11.153	235.355	59.146
Infection (Fig 11) Case Study 3	$R_0 = 2.6$	8840	500	5826.840	11.654	1798.114	1554.886
	$R_0 = 3.07$	8830	500	5630.155	11.260	1741.606	1520.793
	$R_0 = 6.0$	8818	500	4949.313	9.899	1718.252	1480.354
Vaccination (Fig 13 and Table 5) Case Study 3	VaccA	16495	500	8991.828	17.984	2246.849	2057.299
	VaccB	20840	500	11346.181	22.692	2250.278	2318.228
	VaccC	23355	500	16727.571	33.455	2862.153	2512.703
	VaccD	12542	500	7620.026	15.240	1750.923	1883.745
Vaccination and Restriction (Fig 14) Case Study 3	$R_0 = 2.6$; Restriction = 0.50	9011	500	4008.347	8.017	1983.816	1743.626
	$R_0 = 2.6$; Restriction = 0.75	8320	500	4566.876	9.134	1681.857	1472.081
	VaccC; Restriction = 0.50	16722	500	9279.227	18.558	2628.414	2313.595
	VaccC; Restriction = 0.75	15429	500	8139.657	16.279	2540.851	2240.514

4.6 Comments about Multi-resolution Environments

Several points are worth noting when discussing multi-resolution environments. First, in our Case-Scenario (Section 4.1), we defined the points of interest (Ps) as being associated with specific locations within the city of Porto Alegre; consequently, the population motion occurred in the city, with populations moving between Ps . However, the spatial scope of these Ps can be adjusted to suit the needs of simulation designers or specific experiments. For instance, these Ps could represent locations in various cities within a state, and distances between cities could then be calculated, enabling the application of navigation models such as Levy walk or alternative movement algorithms. Movement restrictions could also be modeled, as demonstrated in

Figure 9. Similarly, large-scale events (Section 4.3) could be simulated where individuals travel from one city to another, or regions could be conceptualized as states within a country. This flexibility highlights the adaptability of our population simulation model to operate with different spatial abstractions. Regarding the epidemic contagion scenarios presented in Section 4.4, parameters related to contagion dynamics may require adjustments to reflect larger distances between individuals, such as when simulating interactions across different states. This capability underscores the framework’s scalability and the potential for extending its use to broader contexts.

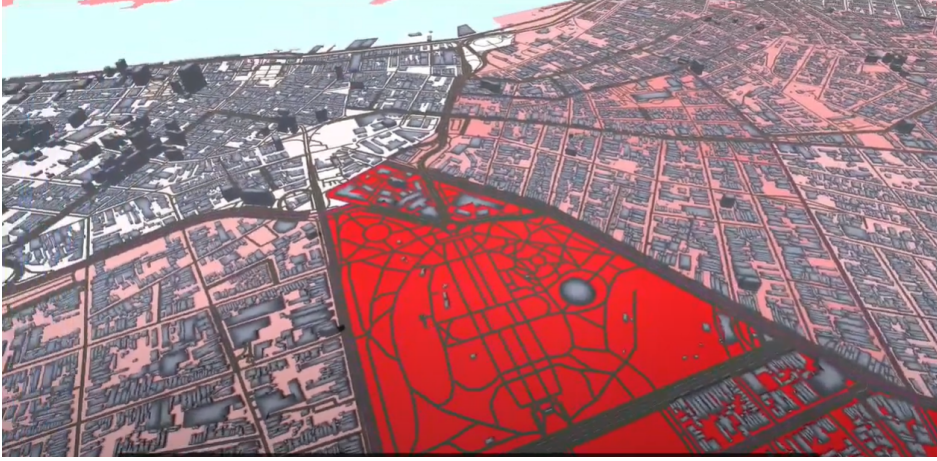
Finally, our simulation results are effectively visualized within multi-level simulation environments, offering insights into population dynamics across different spatial scales. Figure 15 illustrates population movements, highlighting interactions between spatial levels. Figure 15a provides a detailed view of the city environment, incorporating additional visual elements such as roads and buildings to contextualize the simulations. In these visualizations, the red area represents a designated P in the simulation, acting as a focal point for population activities. Figure 15b demonstrates populations moving between multiple P s, showcasing the framework’s ability to model and track mobility across multiple regions. The visualization was developed using Unity.

5 Final Considerations

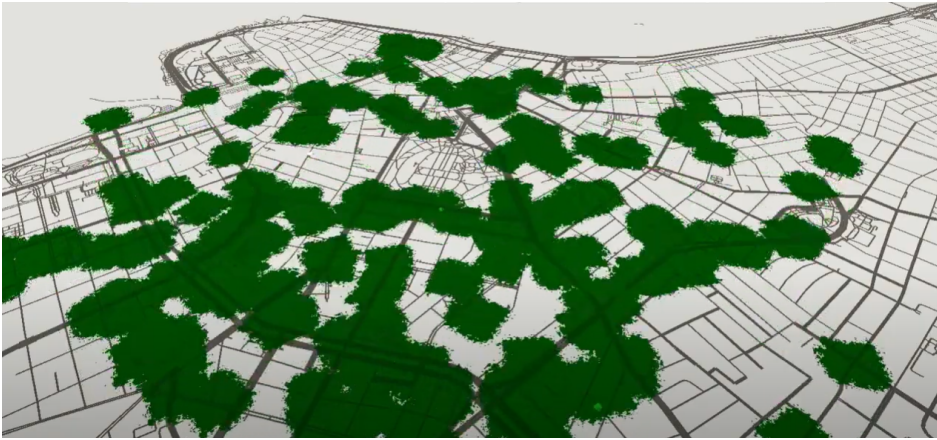
This work introduced LodusPop, a flexible framework for simulating urban population dynamics across multiple spatial and temporal scales. By combining an environment-driven paradigm with blob abstractions, LodusPop enables the representation of population dynamics at multiple spatial and temporal scales while reducing reliance on fine-grained individual data. This makes it particularly suited to contexts where privacy concerns or limited data availability constrain traditional population-driven approaches.

Our experiments demonstrated the framework’s versatility in addressing diverse real-world challenges, ranging from everyday mobility and large-scale events to epidemic contagion and vaccination strategies. These case studies highlight how LodusPop can capture both short-term disruptions and long-term dynamics, underscoring its potential for practical applications in urban planning, public health, disaster management, and crisis response. Importantly, the extensible plugin architecture and JSON-based input design ensure that new action types, data sources, and policies can be integrated with minimal effort, making LodusPop adaptable to different research and policy contexts.

Despite these contributions, some limitations remain. The level of detail in simulations is directly tied to the granularity of available data, often constrained by census or municipal surveys. Current assumptions—such as independence among sampled characteristics—may oversimplify correlations between attributes like age, occupation, or mobility patterns. Addressing these limitations will require integrating dynamic data sources and statistical dependencies, which would further enhance realism and adaptability. Our evaluation reports quantitative runtime and memory measurements for the presented case studies on a single machine; therefore, the results are hardware-



(a) A detailed representation of the city with roads and buildings. The highlighted red area represents a designated P in the environment.



(b) Visualization of populations moving between multiple P s in the environment.

Fig. 15 Visualization of our simulations in a multi-level environment. (a) present a detailed visualization of the city, and (b) presents the abstract population moving between different P s in the environment.

and case-dependent and are intended to characterize scalability trends in our implementation rather than provide head-to-head benchmarking against other frameworks. Moreover, the current implementation operates exclusively with environment-driven blob dynamics; integrating explicit agent-based micro-dynamics via the plugin interface is an important avenue for future work. We also leave as priorities for future work a formal complexity analysis, richer evaluation metrics, and comparative studies against real mobility data. Future research may also explore validation strategies based on synthetic datasets, interoperability with urban digital twin platforms, and coupling with machine learning methods to generate adaptive, data-driven behaviors.

Evaluating LodusPop against state-of-the-art models remains challenging, as existing frameworks like MATSim [40], SUMO [41], and GAMA [42] either require detailed travel demand data, focus narrowly on traffic, or depend on explicit behavioral rules. In contrast, LodusPop offers a complementary perspective: less data-hungry than traffic simulators, more adaptable than statistical mobility models, and uniquely capable of supporting rapid policy evaluation in heterogeneous and low-data environments.

In summary, LodusPop establishes a robust foundation for simulating complex and evolving population dynamics. Its design and extensibility position it as a bridge between research and practice, providing researchers, planners, and policymakers with a powerful tool for evidence-based decision-making in domains as diverse as population mobility, crisis management, and sustainable urban development.

Acknowledgements. The authors acknowledge the use of ChatGPT, version GPT-4o, to review and enhance the text quality of this paper, ensuring clarity and coherence in its presentation.

Declarations

Funding. This study was partly funded by: the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001; the Conselho Nacional de Desenvolvimento Científico e Tecnológico - Brasil (CNPq) - Process Number 309228/2021-2; the Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS) - Process Number 20/2551-0000280-2; and the Instituto Nacional de Ciência e Tecnologia em Simulação e Monitoramento para Assistência ao Indivíduo em Eventos Climáticos Extremos (INCT-SiM-AI) - Process Number 408330/2024-4.

Competing Interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper

Data availability. Data will be made available on request.

Code availability. Code will be made available on request.

Author contribution. **Gabriel Fonseca Silva:** Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing, Visualization. **André da Silva Antonitsch:** Conceptualization, Methodology, Investigation. **Soraia Raupp Musse:** Conceptualization, Methodology, Validation, Investigation, Writing – review & editing, Supervision, Project administration, Funding acquisition.

Appendix A Input and Simulation Setup

This Appendix provides examples of the data input and simulation setup required to execute simulations, as defined by our proposed model in Section 3. The setup includes:

- *i*) input files describing the environment, population, and routines (detailed in Appendices A.1, A.2 and A.3, respectively);

- *ii*) plugin scripts to introduce new action types into the simulation (Appendix A.4); and
- *iii*) a simulator script responsible for executing the simulation.

In the current implementation, both the simulator and plugin scripts are written in Python, and the input data is structured in the JSON file format, ensuring ease of use and extensibility.

A.1 Environment Input Files

The environment input file provides a detailed description of all regions (R s) and their respective points of interest (P s) in the simulation, in accordance with the definitions outlined in Section 3.1. Listing 1 illustrates an example environment configuration containing a single region named “*ExampleRegion*”, which includes three points of interest: a school, a pharmacy, and a home. Data to describe the environment can be obtained from Geographic Information System (GIS) databases such as OpenStreetMap, as presented in Figure 1.

Each element in the environment input file, whether a region or a point of interest, is characterized by an identifier and a geographical location. These attributes are specified using the *name* and *lng_lat* properties, respectively.

```
"regions": [
  {
    "name": "ExampleRegion",
    "lng_lat": [-51.19757, -30.15647],
    "points_of_interest": [
      {
        "name": "school",
        "lng_lat": [-51.19750, -30.15621]
      },
      {
        "name": "pharmacy",
        "lng_lat": [-51.19820, -30.15655]
      },
      {
        "name": "home",
        "lng_lat": [-51.19805, -30.15633]
      }
    ]
  }
]
```

Listing 1 Example of an environment input file.

A.2 Population Input Files

The population input file provides a detailed description of all blobs (B s), including their characteristics and locations at the start of the simulation, in alignment with the definitions in Section 3.2. Listing 2 presents an example population input containing two initial blobs. Population data can be sourced from census data (Real Population) or generated synthetically (Synthetic Population) to meet the specific requirements of an experiment, as shown in Figure 1.

The *default_traceable_characteristics* property sets default values for the T cs of all blobs. Unless explicitly defined in their setup, all blobs start the simulation with a *VaccinationStatus* traceable characteristic set to *Unvaccinated*.

The *sampled_characteristics_categories* property specifies the possible categories for each sampled characteristic used by all blobs in the simulation. However, the population distribution across these categories is defined individually for each blob, as described below.

The *initial_population* property outlines points of interest and their initial populations (i.e., B_P). Points of interest are identified by their containing regions and identifiers, such as *ExampleRegion//home*, which references the *home* P within *ExampleRegion*, as described in Listing 1. A point of interest may include multiple blob descriptions, but only those with an initial population need to be included in the input file.

Each P within *initial_population* contains a vector of elements describing blobs, with each element providing:

- The total population of a blob (TP);
- Optional values for Tc , which override the *default_traceable_characteristics*; and
- Values for all categories of the Sc .

For instance, in Listing 2, two blobs are created in the *home* P of *ExampleRegion*. Blob B_i has $TP_i = 100$, with its Tc_i equal to the default values, as no specific values were assigned in its *traceable_characteristics* property. Blob B_j , on the other hand, has $TP_j = 20$ and an initial *VaccinationStatus* traceable characteristic value of *Vaccinated*.

```
"default_traceable_characteristics": {
  "VaccinationStatus": "Unvaccinated",
  "Traceable_B": "Value_B"
},
"sampling_characteristics_categories": {
  "Age": ["Children", "Adults", "Elders"],
  "Sampled_B": ["Value_B1", "Value_B2"]
},
"initial_population": {
  "ExampleRegion//home": [
    {
      "total_population": 100,
      "traceable_characteristics": {},
      "sampled_characteristics": {
        "Age": { "Children": 20, "Adults": 50, "Elders": 30 },
        "Sampled_B": { "Value_B1": 43, "Value_B2": 57 }
      }
    },
    {
      "total_population": 20,
      "traceable_characteristics": {
        "VaccinationStatus": "Vaccinated"
      },
      "sampled_characteristics": {
        "Age": { "Children": 20, "Adults": 0, "Elders": 0 },
        "Sampled_B": { "Value_B1": 12, "Value_B2": 8 }
      }
    }
  ]
}
```

Listing 2 Example of a population input file.

A.3 Routine Input Files

The routine input file provides a description of the Global Routine (*GRO*) and the local routines for individual points of interest, following the definitions outlined in Section 3.3. Listing 3 presents an example input containing routines for two points of interest: a *home* P_i and a *pharmacy* P_j , both located in the *ExampleRegion*.

P_i includes a single action, A_i , executed at $t = 8$ during each simulation cycle. A_i is of type *move population* (defined in Section 3.3) and moves 20 individuals—specifically unvaccinated children and young students—from P_i to the *pharmacy* P_j in *ExampleRegion*.

P_j , on the other hand, has two actions, A_j and A_k , performed at $t = 9$ and $t = 10$, respectively, during each simulation cycle. A_j is of type *change traceable characteristic* (detailed in Section 3.3), vaccinating 5 people currently at P_j . A_k , of type *move population*, transfers the vaccinated individuals from P_j back to P_i .

The *population_template* property specifies the profile of individuals selected for each action, such as students who are children or young adults. The *name* property, included as a label, does not influence the simulation directly but serves as a reference for annotating the purpose of the action. While Listing 3 does not include actions for the Global Routine, these follow the same structure as individual routines, encompassing a cycle step, action type, population template, and any additional parameters.

```
"global_routine": [],
"routines": {
  "ExampleRegion//home": [
    {
      "cycle_step": 8,
      "action": {
        "name": "send_students_to_pharmacy",
        "type": "move_population",
        "population_template": {
          "traceable_characteristics": {
            "VaccinationStatus": "Unvaccinated"
          },
          "sampled_characteristics": {
            "Age": [ "Children", "Youngs" ],
            "Occupation": [ "Student" ]
          }
        },
        "values": {
          "origin_region": "ExampleRegion",
          "origin_poi": "home",
          "destination_region": "ExampleRegion",
          "destination_poi": "pharmacy",
          "quantity": 20
        }
      }
    }
  ],
  "ExampleRegion//pharmacy": [
    {
      "cycle_step": 9,
      "action": {
        "name": "vaccinate_population",
        "type": "change_traceable_characteristic",
        "population_template": {
          "traceable_characteristics": {
            "VaccinationStatus": "Unvaccinated"
          },
          "sampled_characteristics": {}
        },
        "values": {
          "characteristic_to_change": "VaccinationStatus",
          "new_value": "Vaccinated",
          "quantity": 5
        }
      }
    },
    {
      "cycle_step": 10,
      "action": {
        "name": "send_vaccinated_population_back",
```

```

        "type": "move-population",
        "population_template": {
            "traceable_characteristics": {
                "VaccinationStatus": "Vaccinated"
            },
            "sampled_characteristics": {}
        },
        "values": {
            "origin_region": "ExampleRegion",
            "origin_poi": "pharmacy",
            "destination_region": "ExampleRegion",
            "destination_poi": "home",
            "quantity": 5
        }
    }
}
]
}

```

Listing 3 Example of a routine input file.

A.4 Plugins

Plugins define additional action types and their corresponding functions while also providing external parameters and data. These resources enable simulation designers to tailor simulations and model diverse scenarios. Plugins are implemented as Python modules and imported by the simulator script, described in the following section.

Listing 4 shows an example plugin that processes the *"example_action"* action type. This action transfers 100 individuals from a point of interest P_i , specified by the *values* parameter, to a *home* P_j within the same region. The plugin accomplishes this by creating and returning a *move population* action, which is then executed by its corresponding function.

```

class ExamplePlugin(environment.TimeActionPlugin):

    def __init__(self, environment_graph):
        super().__init__()
        self.graph = environment_graph
        self.set_pair('example_action', self.consume_action)
        self.example_external_parameter = 'foo'

    def consume_action(self, pop_template, values, cycle_step, sim_step):
        region = self.graph.get_region(values['region'])
        poi = region.get_poi(values['node'])

        new_action_type = 'move-population'
        new_action_values = {}
        new_action_values['origin_region'] = region.name
        new_action_values['origin_poi'] = poi.name
        new_action_values['destination_region'] = region.name
        new_action_values['destination_poi'] = 'home'
        new_action_values['quantity'] = 100
        action = environment.TimeAction(_type = new_action_type,
                                         _pop_template = pop_template,
                                         _values = new_action_values)

        return [action]

```

Listing 4 Example of a plugin script that includes a *"example_action"* in the simulation.

A.5 Simulator script

The simulator script manages the parsing of input data, the integration of plugins, and the overall control of the simulation flow. Listing 5 presents an example of a simulator script. Parsing is handled by a function that takes input file paths as arguments and

returns an *EnvGraph* framework class instance. Plugins are incorporated by initializing their classes and loading them into the *EnvGraph*. The simulation flow is governed by a loop that updates the simulation step at each iteration.

```
# Data Loading
env_graph = Generate.EnvironmentGraph([data_input_file_paths])

# Simulation Parameters
cycles = 3
cycle_duration = 24
env_graph.routine_cycle_length = cycle_duration
simulation_steps = cycles * cycle_duration

# Loading Plugins
ex_plugging = ExamplePlugin(env_graph)
ex_plugging.example_external_parameter = 'bar'
env_graph.LoadPlugin(ex_plugging)

# SIMULATION
for i in range(simulation_steps):
    # Updates POI Routines and Global Routines
    # These are defined by the input files
    env_graph.update_time_step(i % cycle_duration, I)

    # Actions can be directly called or queued in the simulation loop
    # if i == 60:
    #     dummy_action = TimeAction(_type = new_action_type,
    #                               _pop_template = dummy_template,
    #                               _values = dummy_values)
    #     env_graph.direct_action_invoke(dummy_action)
    #     env_graph.queue_next_frame_action(dummy_action)
```

Listing 5 Simulation script example.

References

- [1] Sharifi, A., Khavarian-Garmsir, A.R.: The covid-19 pandemic: Impacts on cities and major lessons for urban planning, design, and management. *Science of The Total Environment* **749**, 142391 (2020) <https://doi.org/10.1016/j.scitotenv.2020.142391>
- [2] Kang, B.G., Park, H.-M., Jang, M., Seo, K.-M.: Hybrid Model-Based simulation analysis on the effects of social distancing policy of the COVID-19 epidemic. *Int J Environ Res Public Health* **18**(21) (2021)
- [3] Bouchnita, A., Chekroun, A., Jebrane, A.: Mathematical modeling predicts that strict social distancing measures would be needed to shorten the duration of waves of COVID-19 infections in vietnam. *Front Public Health* **8**, 559693 (2021)
- [4] Eid, M.S., Eldin, H.K.: Simulation models as a tool for urban planning. *Computers & Industrial Engineering* **4**(1), 53–66 (1980) [https://doi.org/10.1016/0360-8352\(80\)90039-X](https://doi.org/10.1016/0360-8352(80)90039-X)
- [5] Koumetio Tekouabou, S.C., Diop, E.B., Azmi, R., Jaligot, R., Chenal, J.: Reviewing the application of machine learning methods to model urban form indicators in planning decision support systems: Potential, issues and challenges. *Journal*

- [6] Haraguchi, M., Nishino, A., Kodaka, A., Allaire, M., Lall, U., Kuei-Hsien, L., Onda, K., Tsubouchi, K., Kohtake, N.: Human mobility data and analysis for urban resilience: A systematic review. *Environment and Planning B: Urban Analytics and City Science* **49**(5), 1507–1535 (2022) <https://doi.org/10.1177/23998083221075634>
- [7] Serena, L., Marzolla, M., D’Angelo, G., Ferretti, S.: A review of multilevel modeling and simulation for human mobility and behavior. *Simulation Modelling Practice and Theory* **127**, 102780 (2023) <https://doi.org/10.1016/j.simpat.2023.102780>
- [8] Helbing, D., Molnar, P.: Social force model for pedestrian dynamics. *Physical Review E* **51**(5), 4282 (1995)
- [9] Cao, R.F., Lee, E.W.M., Yuen, A.C.Y., Chen, T.B.Y., De Cachinho Cordeiro, I.M., Shi, M., Wei, X., Yeoh, G.H.: Simulation of competitive and cooperative egress movements on the crowd emergency evacuation. *Simulation Modelling Practice and Theory* **109**, 102309 (2021) <https://doi.org/10.1016/j.simpat.2021.102309>
- [10] Khan, S., Deng, Z.: Agent-based crowd simulation: an in-depth survey of determining factors for heterogeneous behavior. *The Visual Computer* **40**(7), 4993–5004 (2024) <https://doi.org/10.1007/s00371-024-03503-2>
- [11] Ruprecht, I., Michelic, F., Eggeling, E., Preiner, R.: Adaptive movement behavior for real-time crowd simulation. *The Visual Computer* **40**(7), 4789–4803 (2024) <https://doi.org/10.1007/s00371-024-03476-2>
- [12] Hughes, R.L.: A continuum theory for the flow of pedestrians. *Transportation Research Part B: Methodological* **36**(6), 507–535 (2002)
- [13] Treuille, A., Cooper, S., Popović, Z.: Continuum crowds. *ACM Transactions on Graphics* **25**(3), 1160–1168 (2006)
- [14] Thalmann, D., Musse, S.R.: *Crowd Simulation*. Springer, London (2007). <https://doi.org/10.1007/978-1-84628-825-8>
- [15] Xiong, M., Lees, M., Cai, W., Zhou, S., Low, M.Y.H.: Hybrid modelling of crowd simulation. *Procedia Computer Science* **1**(1), 57–65 (2010)
- [16] Ijaz, K., Sohail, S., Hashish, S.: A survey of latest approaches for crowd simulation and modeling using hybrid techniques. In: *Proceedings of the 2015 17th UKSIM-AMSS International Conference on Modelling and Simulation. UKSIM ’15*, pp. 111–116. IEEE Computer Society, USA (2015)

- [17] Antonitsch, A.D.S., Schaffer, D.H.M., Rockenbach, G.W., Knob, P., Musse, S.R.: BioClouds: A multi-level model to simulate and visualize large crowds. In: Computer Graphics International Conference, pp. 15–27 (2019). Springer
- [18] Dang, H.-T., Gaudou, B., Verstaavel, N.: A multi-level density-based crowd simulation architecture. In: Mathieu, P., Dignum, F., Novais, P., Prieta, F. (eds.) *Advances in Practical Applications of Agents, Multi-Agent Systems, and Cognitive Mimetics*. The PAAMS Collection, pp. 64–75. Springer, Cham (2023)
- [19] Cogo, E., Prazina, I., Hodzic, K., Haseljc, H., Rizvic, S.: Survey of integrability of procedural modeling techniques for generating a complete city. In: 2019 XXVII International Conference on Information, Communication and Automation Technologies (ICAT), pp. 1–6 (2019). <https://doi.org/10.1109/ICAT47117.2019.8939012>
- [20] Alessandretti, L., Aslak, U., Lehmann, S.: The scales of human mobility. *Nature* **587**(7834), 402–407 (2020) <https://doi.org/10.1038/s41586-020-2909-1>
- [21] Van Den Berg, J., Guy, S.J., Lin, M., Manocha, D.: Reciprocal n-Body Collision Avoidance. In: *Robotics Research*, pp. 3–19. Springer, ??? (2011)
- [22] Lima Bicho, A., Rodrigues, R.A., Musse, S.R., Jung, C.R., Paravisi, M., Magalhães, L.P.: Simulating crowds based on a space colonization algorithm. *Computers & Graphics* **36**(2), 70–79 (2012)
- [23] Pelechano Gómez, N., O’Brien, K., Silverman, B.G., Badler, N.: Crowd simulation incorporating agent psychological models, roles and communication. In: *First International Workshop on Crowd Simulation*, pp. 21–31 (2005)
- [24] Durupinar, F., Gdkbay, U., Aman, A., Badler, N.I.: Psychological parameters for crowd simulation: From audiences to mobs. *IEEE Transactions on Visualization and Computer Graphics* **22**(9), 2145–2159 (2015)
- [25] Neto, A.B.F., Pelachaud, C., Musse, S.R.: Giving emotional contagion ability to virtual agents in crowds. In: *International Conference on Intelligent Virtual Agents*, pp. 63–72 (2017). Springer
- [26] Antonitsch, A., Musse, S.R., Figueiredo, L.H.: Towards a legion of virtual humans: Steering behaviors and organic visualization. In: *SIBGRAPI Conference on Graphics, Patterns and Images*, pp. 31–38 (2020). IEEE
- [27] Lemonari, M., Panayiotou, A., Kyriakou, T., Pelechano, N., Chrysanthou, Y., Aristidou, A., Charalambous, P.: Mpact: Mesoscopic profiling and abstraction of crowd trajectories. *Computer Graphics Forum* **n/a**(n/a), 70156 <https://doi.org/10.1111/cgf.70156> <https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.70156>

- [28] Alvarez, H.V., Patsatzis, D.G., Russo, L., Kevrekidis, I., Siettos, C.: Next Generation Equation-Free Multiscale Modelling of Crowd Dynamics via Machine Learning (2025). <https://arxiv.org/abs/2508.03926>
- [29] Li, Y., Chen, Y., Liu, J., Huang, T.: Efficient crowd simulation in complex environment using deep reinforcement learning. *Scientific Reports* **15**(1), 5403 (2025) <https://doi.org/10.1038/s41598-025-88897-2>
- [30] Brockmann, D., Hufnagel, L., Geisel, T.: The scaling laws of human travel. *Nature* **439**(7075), 462–465 (2006)
- [31] Gonzalez, M.C., Hidalgo, C.A., Barabasi, A.-L.: Understanding individual human mobility patterns. *Nature* **453**(7196), 779–782 (2008)
- [32] Song, C., Koren, T., Wang, P., Barabási, A.-L.: Modelling the scaling properties of human mobility. *Nature Physics* **6**(10), 818–823 (2010) <https://doi.org/10.1038/nphys1760>
- [33] Jamshidi, S., Baniasad, M., Niyogi, D.: Global to USA county scale analysis of weather, urban density, mobility, homestay, and mask use on COVID-19. *International Journal of Environmental Research and Public Health* **17**(21), 7847 (2020)
- [34] Hernández-Orallo, E., Armero-Martínez, A.: How human mobility models can help to deal with COVID-19. *Electronics* **10**(1), 33 (2021)
- [35] Kraemer, M.U.G., Yang, C.-H., Gutierrez, B., Wu, C.-H., Klein, B., Pigott, D.M., Plessis, L., Faria, N.R., Li, R., Hanage, W.P., Brownstein, J.S., Layan, M., Vespignani, A., Tian, H., Dye, C., Pybus, O.G., Scarpino, S.V.: The effect of human mobility and control measures on the covid-19 epidemic in china. *Science* **368**(6490), 493–497 (2020) <https://doi.org/10.1126/science.abb4218>
- [36] Silva, G.F., Cassol, V., Neto, A.B.F., Antonitsch, A., Schaffer, D., Musse, S.R., Marsillac Linn, R.: LODUS: A multi-level framework for simulating environment and population - A contagion experiment on a pandemic world. In: *IEEE International Smart Cities Conference*, pp. 1–8 (2020). IEEE
- [37] Liu, J., Xu, L., Ma, L., Chen, N.: Modeling population mobility flows: A hybrid approach integrating a gravity model and machine learning. *ISPRS International Journal of Geo-Information* **13**(11) (2024) <https://doi.org/10.3390/ijgi13110379>
- [38] Guo, H., Zhang, W., Yang, J., Hou, Y., Dong, L., Liu, Y.: Data driven discovery of human mobility models (2025). <https://arxiv.org/abs/2501.05684>
- [39] Du, Y., Feng, J., Yuan, J., Li, Y.: CAMS: A CityGPT-Powered Agentic Framework for Urban Human Mobility Simulation (2025). <https://arxiv.org/abs/2506.13599>

- [40] Balmer, M., Rieser, M., Meister, K., Charypar, D., Lefebvre, N., Nagel, K.: Matsim-t: Architecture and simulation times. In: Multi-agent Systems for Traffic and Transportation Engineering, pp. 57–78. IGI Global Scientific Publishing, Hershey, PA (2009)
- [41] Krajzewicz, D., Erdmann, J., Behrisch, M., Bieker-Walz, L.: Recent development and applications of sumo - simulation of urban mobility. *International Journal On Advances in Systems and Measurements* **3 and 4** (2012)
- [42] Taillandier, P., Gaudou, B., Grignard, A., Huynh, Q.-N., Marilleau, N., Caillou, P., Philippon, D., Drogoul, A.: Building, composing and experimenting complex spatial models with the gama platform. *GeoInformatica* **23**(2), 299–322 (2019) <https://doi.org/10.1007/s10707-018-00339-6>
- [43] Kermack, W.O., McKendrick, A.G.: A contribution to the mathematical theory of epidemics. *Proceedings of the royal society of london. Series A, Containing papers of a mathematical and physical character* **115**(772), 700–721 (1927)
- [44] Dutra, C.: Estimate of the basic number of reproduction r_0 of covid-19 in south american countries. *InterAmerican Journal of Medicine and Health* **3**, 1–7 (2020) <https://doi.org/10.31005/iajmh.v3i0.127>
- [45] Souza, W.M., Buss, L.F., Candido, D.d.S.: Epidemiological and clinical characteristics of the covid-19 epidemic in brazil. *Nature Human Behaviour* **4**(8), 856–865 (2020) <https://doi.org/10.1038/s41562-020-0928-4>
- [46] Franco, C.M.R., Dutra, R.F.: Modelo sir para propagação da covid-19 no estado da paraíba (brasil). *INTERMATHS* **2**(2), 39–48 (2021) <https://doi.org/10.22481/intermaths.v2i2.9696>